

Performance of Rust language

May 2026

About us



Yuri Gribov (yugr, the_real_yugr), <https://github.com/yugr>

- Developer and fan of system SW (compilers, runtimes, verification tools, etc.)
- Compiler teamlead



Zakhar Akimov, <https://github.com/z-inform>

- Developer of general-purpose and AI compilers
- ML-in-compilers expert

Rust language features

- Language for high-performance system programming
 - Zero cost/overhead abstractions: don't pay for what you don't use + no (or minimal) runtime overhead (iterators, closures, traits, etc.)
 - Allows low-level tuning (SIMD, inline assembler, compiler intrinsics like `__builtin_expect`, etc.)
- Supports modern programming paradigms
 - Procedural, functional and object-oriented programming
 - Static and runtime polymorphism
 - Metaprogramming
- Reduces number of UBs through stricter language rules and runtime checks
 - Especially memory safety

Rust performance: Disadvantages



- Runtime checks
- Forced initialization
- Integer overflows are defined behavior
- Performance-robustness tradeoffs in stdlib
- ...

Rust performance: Advantages



- More aggressive alias analysis
 - Basically restrict-by-default
- Structure layout optimizations
 - Field reorder and niche optimizations
- More efficient standard library containers
- ...

Goals of this talk



<https://pxhere.com/ru/photo/870592>

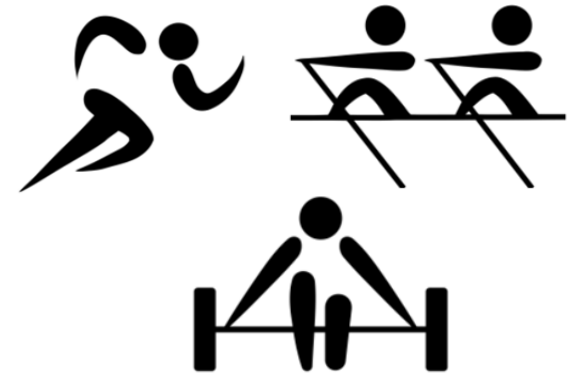
- How “expensive” are Rust checks in practice?
 - Compared to Rust without checks and/or C/C++
 - What are the immediate (extra instructions) and indirect (disabling optimizations, I\$/BTB pressure) consequences
- Can Sufficiently Smart (Heroic) Compilers © help to mitigate overheads?
- And if not, can developer help compiler to get rid of them?
- On the contrary, what language features help compiler to optimize code better?

Opinion of language creators

- [Steven Klabnik](#):
 - "Generally Rust is the same order of magnitude as C. Sometimes faster, sometimes slower"
 - "Rust prioritizes memory safety above all else. But speed is a close second"

Overhead analysis

- How can we estimate overheads caused by checks?
- Rust compiler does not provide flags to disable safety checks
- We will compare regular Rust with [patched Rust](#) with disabled checks
 - Repo with modified compiler: [yugr/rust-private](#)
 - Baseline branch [yugr/baseline](#) (on top of 1.87.0)
 - Disabled hashtable randomization and forced function alignment for better measurement stability



Benchmarks

- Rust compiler runtime benchmarks
- Benchmarks from large open-source Rust projects from various domains:
 - Gamedev (Bevy, Veloren)
 - Databases (SpacetimeDB)
 - Text processing (Zed, Ruff, regex)
 - Codecs (rav1e, oxipng)
 - Linear algebra (nalgebra)
 - Asynchronous runtime (tokio)
 - Packet managers (uv)
- Many benchmarks have already been optimized (via unsafe, etc.)

Rust and C++



- Rust and C++ are similar in expressive power:
 - Support several programming paradigms (procedural, OOP, functional)
 - Different types of polymorphism
 - Etc.
- Similar application domain (high-performance system programming)
 - Zero cost/overhead abstractions
 - Support for low-level tuning
- Same backend optimizers (LLVM, GCC) and optimization technology (PGO, LTO, etc.)
- “Safe” C++ dialect (Hardened C/C++) has similar runtime checks

Not covered: Other language aspects

- Ergonomics, expressiveness (e.g. self-referential data structures), safety, etc.
- A lot of talks on these topics already!
 - [Rust Features that I Want in C++](#) (David Sankel, CppNow 2022)
 - [The existential threat against C++ and where to go from here](#) (Helge Penne, NDC 2024)
 - [Rust vs C++ with Steve Klabnik and Herb Sutter](#) (SW Engineering Daily podcast)

Not covered: Parallel programming

- Easy and reliable parallel programming is considered one of the big advantages of Rust
 - [Fearless Concurrency with Rust](#)
- But this requires a separate talk and deeper expertise
 - RustCon 2027 anyone?

Not covered: Unsafe code



- Rust language consists of two parts:
 - Safe code (most of Rust code)
 - Unsafe code:
 - “Things the Rust authors will implore you not to do” ([Rustonomicon](#))
 - “Aspects of the language you may not use every day and useful in very specific situations” ([Rust Book](#))
 - Used for hotspot optimizations
 - Used for tuning in real-world projects (including benchmarks that we consider)
- In this talk we focus on *safe* code optimizations:
 - (Also not covering third-party libraries, SIMD and compiler intrinsics)
 - Unsafe constructs remove the main advantage of the language
 - Unsafe optimizations are well-known and often trivial

Not covered: Non-idiomatic code

- [Nikita Popov](#), LLVM lead maintainer:
 - “Languages like C, C++ and Rust are by-design on equal footing”
 - “You can only compare how easy it is to write performant code or how performant idiomatic code is”
- [Scott McMurray](#), Rust compiler team:
 - “Clang and rustc are both producing LLVM IR and running mostly the same LLVM optimization passes”
 - “Given enough work they can always produce essentially the same thing, making the comparisons pointless”

Bounds checks

Buffer overflows

- Morris Worm (1988)
- Writing excessively large volume of data into program variable

```
char local_buf[32];  
sprintf(buf, "Message from user: %s", received_data);
```

- Stack overflow
 - Stack Smashing, Return-to-libc, Return-Oriented Programming attacks
 - The most standardized and widely used type of attacks
- Heap overflow



<https://www.rawpixel.com/image/5958324/free-public-domain-cc0-photo>

Prevalence of buffer overflow vulnerabilities



- Leads ratings of the most dangerous vulnerabilities
 - [Mitre CVE Top 25 2024](#) (2, 6, 20 places)
- 70% vulnerabilities in [Microsoft](#) products and [Chromium](#) caused by memory errors
- And up to 80% memory errors are caused buffer overflow
 - [CVE](#) and [KEV](#) statistics (2024)
 - [Google Project Zero](#) (2024)

Memory safety in Rust

- Frontend generates bounds checks for standard index accesses
 - Arrays, slices, containers, etc.
 - Checks may be eliminated later if optimizer can guarantee correctness
- Uses safer algorithms in stdlib
 - E.g. sorting won't crash on incorrect comparators



Overheads

- Comparison and non-taken conditional jump (1-2 ALU slots)
- Extra register for keeping object length for comparison
- I\$ pressure (cache misses) due to checking code and panic handlers
 - Branch predictor pressure for never-taken branches is [likely none](#)
- Disabling other optimizations
 - Mainly vectorizer

Compiler optimizations

- In many cases compiler can
 - Prove correctness of memory access and remove checks
 - Or extract checks from loops ($O(N)$ $\rightarrow$ $O(1)$)
- Such optimizations are done in LLVM optimizer
 - InstCombine, SimpleLoopUnswitch, LoopVectorize, etc.
 - Also help Hardened C/C++
- Optimizer has been significantly improved and many older issues are successfully optimized in modern Rust
 - E.g. well-known [examples of Alex Kladov](#) (matklad)

Successful optimization

All checks moved to
function prologue

```
pub fn foo(a: &[i32], b: &[i32]) -> i32 {  
    let mut ans = 0;  
    let n = a.len();  
    for i in 0..n {  
        ans += a[i] * b[i];  
    }  
    ans  
}
```

<https://godbolt.org/z/aY744zdfe>



Compiled with -Cno-
vectorize-loops
-Cno-vectorize-slp
-Cllvm-args=-unroll-
allow-remainder=0

foo:

```
.cfi_startproc  
test    rsi, rsi  
je      .LBB0_1 // n == 0 => return 0  
lea     rax, [rsi - 1]  
cmp     rcx, rax  
jbe     .LBB0_6 // b.len() < n => panic  
xor     eax, eax  
xor     ecx, ecx  
.p2align    4  
.LBB0_4: // Loop (without checks!)  
mov     r8d, dword ptr [rdx + 4*rcx]  
imul    r8d, dword ptr [rdi + 4*rcx]  
lea     r9, [rcx + 1]  
add     eax, r8d  
mov     rcx, r9  
cmp     rsi, r9  
jne     .LBB0_4
```

...

Unsuccessful optimization

Checks remain within the loop

```
pub fn foo(v: &[i32]) -> i32 {  
    let mut ans = 0;  
    let chunk_len = 4;  
    let num_chunks = v.len() / chunk_len;  
    for i in 0..num_chunks {  
        for j in 0..chunk_len {  
            ans += v[i * chunk_len + j];  
        }  
    }  
    ans  
}
```



<https://godbolt.org/z/176s9nEfE>

Compiled with -Cno-
vectorize-loops
-Cno-vectorize-slp
-Cllvm-args=-unroll-
allow-remainder=0

```
bar:  
    ...  
    .p2align    4  
.LBB0_2:    // Loop (with checks!)  
    lea        rcx, [r8 - 3]  
    cmp        rcx, rsi  
    jae        .LBB0_7  
    lea        rcx, [r8 - 2]  
    cmp        rcx, rsi  
    jae        .LBB0_7  
    lea        rcx, [r8 - 1]  
    cmp        rcx, rsi  
    jae        .LBB0_7  
    cmp        r8, rsi  
    jae        .LBB0_6  
    add        eax, dword ptr [rdi + 4*r8 - 12]  
    add        eax, dword ptr [rdi + 4*r8 - 8]  
    add        eax, dword ptr [rdi + 4*r8 - 4]  
    add        eax, dword ptr [rdi + 4*r8]  
    add        r8, 4  
    dec        rdx  
    jne        .LBB0_2
```

Problem with optimizer: delayed panics

- [Rust RFC 560](#) suggested that compiler could combine several checks together
 - If it helps optimization
 - And observed program behavior won't change (modulo error message)
- Similar to classical concept of [Imprecise Exceptions](#)
- This has not been decided and optimization currently isn't enabled for bounds checks ☹️
- Example: [rust #50759](#)

```
use std::fs::{read, write};

fn main() -> Result<(), std::io::Error> {
    let h = read("/tmp/test"?);
    // Each access checked separately ☹️
    // (would be enough to check only h[15])
    let b = [h[0], h[1], ... h[15]];
    write("/tmp/test", &b)?;
    Ok(())
}
```

Workarounds: Overview

- Use slices (rather than containers like Vec) in loops
 - This simplifies job of alias analysis
 - Example: [fschutt/fastblur #3](#)
- Avoid complex index computations
 - Anything more complicated than adding a constant or adding two variables
 - Even affine operations may break optimizer: [Taking Advantage of Auto-Vectorization in Rust](#)
- Do not rely on optimizer in non-trivial cases
 - Optimizations vary (non-monotonically) across compiler versions
 - [“Auto-vectorization is not a programming model”](#)
- Same advices are relevant for Hardened C/C++!

Workarounds

- Checks elimination
- Reducing number of checks
- Reducing cost of checks

Workarounds

- **Checks elimination**
- Reducing number of checks
- Reducing cost of checks

Workarounds: Unsafe-code

- The simplest and most robust solution
- Used in stdlib containers and iterators
- Several variants:
 - Explicit pointers in unsafe blocks
 - Check-less APIs like `get_unchecked`, etc.
 - Compiler hints `std::hint::unreachable_unchecked` and `std::hint::assert_unchecked`



```
fn drop(&mut self) {  
    // fill the hole again  
    unsafe {  
        let pos = self.pos;  
        ptr::write(  
            self.data.get_unchecked_mut(pos),  
            self.elt.take().unwrap());  
    }  
}
```

[rust #36072](#)

Workarounds: Bounded types

- Index range can be expressed via type system:
 - Enums or bool
- Example: [rust #102303](#)

```
pub fn foo(array: &[i16; 3],  
           i: usize) -> i16 {  
    array[i]  
}
```



```
pub enum Enum {  
    A, B, C  
}  
  
pub fn foo(array: &[i16; 3],  
           i: Enum) -> i16 {  
    array[i as usize]  
}
```

Workarounds

- Checks elimination
- **Reducing number of checks**
- Reducing cost of checks

Workarounds: Manual asserts

- Replaces many checks with one
- Example: [jpeg-decoder #167](#)

```
// optimizer hint to eliminate bounds checks
// within loops
assert!(coefficients.len() == 64);

let mut temp = [Wrapping(0); 64];

// columns
for i in 0..8 {
    if coefficients[i + 8] == 0
        && coefficients[i + 16] == 0
        && coefficients[i + 24] == 0
        && coefficients[i + 32] == 0
        && coefficients[i + 40] == 0
        && coefficients[i + 48] == 0
        && coefficients[i + 56] == 0
    {
        ...
    }
}
```

Workarounds: Reslicing

- Reslicing, preslicing or subslicing
- Pre-construct slices of known size
- Forces compiler to perform *single* check when creating slice
- Example: [dropbox/rust-brotli](https://github.com/dropbox/rust-brotli)



```
for i in 0..vec.len() {  
    black_box(vec[i]);  
}  
→  
let slice = &vec[0..vec.len()];  
for i in 0..vec.len() {  
    black_box(slice[i]);  
}
```

Workarounds: Reslicing

- Offsetted slices should better be constructed in two steps to avoid potential integer overflow
- Example: [Optimizing rav1d, an AV1 Decoder in Rust](#)

`&vec[i..(i + n)]`  `&vec[i..][..n]`

Workarounds: Limiting iteration range

- When iterating over several containers explicitly compute common safe range

```
pub fn foo(x: &mut [i32], y: &[i32]) {  
    let n = x.len();  
    for i in 0..n {  
        x[i] ^= y[i];  
    }  
}
```

<https://godbolt.org/z/P8de874c1>

```
.LBB1_2:  
    cmp rcx, rax  
    je .LBB1_5  
    mov r8d, dword ptr [rdx + 4*rax]  
    xor dword ptr [rdi + 4*rax], r8d  
    lea r8, [rax + 1]  
    mov rax, r8  
    cmp rsi, r8  
    jne .LBB1_2
```



```
pub fn foo(x: &mut [i32], y: &[i32]) {  
    let n = std::cmp::min(x.len(), y.len());  
    for i in 0..n {  
        x[i] ^= y[i];  
    }  
}
```

<https://godbolt.org/z/6TzEPaaEs>

```
.LBB0_2:  
    mov     ecx, dword ptr [rdx + 4*rax]  
    xor     dword ptr [rdi + 4*rax], ecx  
    lea     rcx, [rax + 1]  
    mov     rax, rcx  
    cmp     rsi, rcx  
    jne     .LBB0_2
```

Compiled with
-Cno-vectorize-loops -Cllvm-args=-unroll-allow-remainder=0

Workarounds: Iterators

- Many iterators know container size and can utilize this information to remove bounds checks

```
pub fn foo(x: &mut [i32], y: &[i32]) {  
    let n = x.len();  
    for i in 0..n {  
        x[i] ^= y[i];  
    }  
}
```

```
.LBB1_2:                                     https://godbolt.org/z/  
P8de874c1  
    cmp rcx, rax  
    je .LBB1_5  
    mov r8d, dword ptr [rdx + 4*rax]  
    xor dword ptr [rdi + 4*rax], r8d  
    lea r8, [rax + 1]  
    mov rax, r8  
    cmp rsi, r8  
    jne .LBB1_2
```



```
pub fn foo(x: &mut [i32], y: &[i32]) {  
    // This also works:  
    // for (x, y) in x.iter_mut().zip(y)  
    x.iter_mut()  
        .zip(y)  
        .for_each(|(x, y)| *x ^= *y);  
}
```

```
.LBB0_2:                                     https://godbolt.org/z/  
je69173xv  
    mov     ecx, dword ptr [rdx + 4*rax]  
    xor     dword ptr [rdi + 4*rax], ecx  
    lea     rcx, [rax + 1]  
    mov     rax, rcx  
    cmp     rsi, rcx  
    jne     .LBB0_2
```

Compiled with
-Cno-vectorize-loops -Cllvm-args=-unroll-allow-remainder=0

Disadvantages of iterators



- “Miracle of zero cost abstraction has a limit” ([fridsun at Reddit](#))
- Significant slowdown and code size increase for debug builds
- Compiler sometimes fails to inline long iterator chains
- Iterator lambdas take references to container elements which may cause issues with alias analysis ([rust #106539](#))
- Chained iterators (chain, flat_map, flatten, ..=, etc.) may generate inefficient code in for-loops (“external iteration”)
 - Use only with .for_each, .fold, .find, etc. (“internal iteration”)
 - [Are iterators even efficient?](#)

Workarounds: Order of accesses

- Prefer to access data starting with farthest element

```
let a = x[0];  
let b = x[1];  
let c = x[2];
```



```
let c = x[2];  
let b = x[1];  
let a = x[0];
```

or

```
let a = x[0];  
let b = x[1];  
let c = x[2];
```



```
let [a, b, c] = data[..3] else {  
    panic!()  
};
```

or

```
let a = x[0];  
let b = x[1];  
let c = x[2];
```



```
let [a, b, c] = data[..3]  
    .try_into().unwrap();
```

Workarounds: SIMD

- Developer can manually vectorize code
- Stdlib provides
 - Portable `std::simd` API (nightly only)
 - Platform vector intrinsics `std::arch (unsafe)`
- There are also many third-party libraries

```
pub fn sum_simd(data: &[i32]) -> i32 {  
    const LANES: usize = 8; // AVX  
  
    let mut acc = i32x8::splat(0);  
  
    for chunk in data.chunks_exact(LANES) {  
        let v = i32x8::from_slice(chunk);  
        acc += v;  
    }  
  
    let mut tail_acc = 0i32;  
    for &x in data.chunks_exact(LANES).remainder() {  
        tail_acc += x;  
    }  
  
    acc.reduce_sum() + tail_acc  
}
```

<https://godbolt.org/z/G3bfGEhnG>

Workarounds

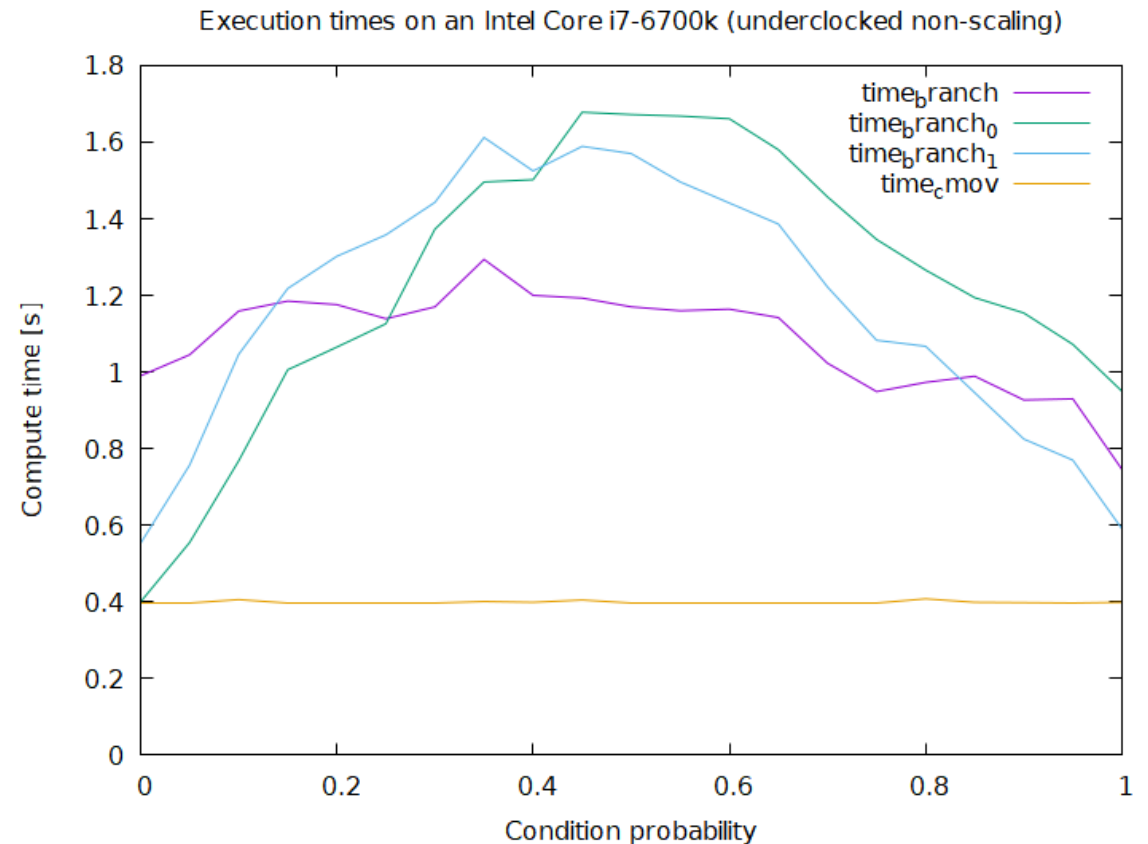
- Checks elimination
- Reducing number of checks
- **Reducing cost of checks**

Workarounds: Arithmetic instead of checks

- Checks may be replaced with conditional move

```
let i = cmp::min(i, sums.len() - 1);  
sums[i] = ...
```

- CMOV may be faster or slower than branch
- Disadvantage: bounds errors will now be ignored



<https://github.com/marcin-osowski/cmov>

Workarounds: Arithmetic instead of checks

- Bounds checks may be replaced with masking if length is 2^N (or can be padded to 2^N):

```
const SIZE: usize = 100;  
const LOOKUP_TABLE_SIZE: usize = SIZE.next_power_of_two();
```

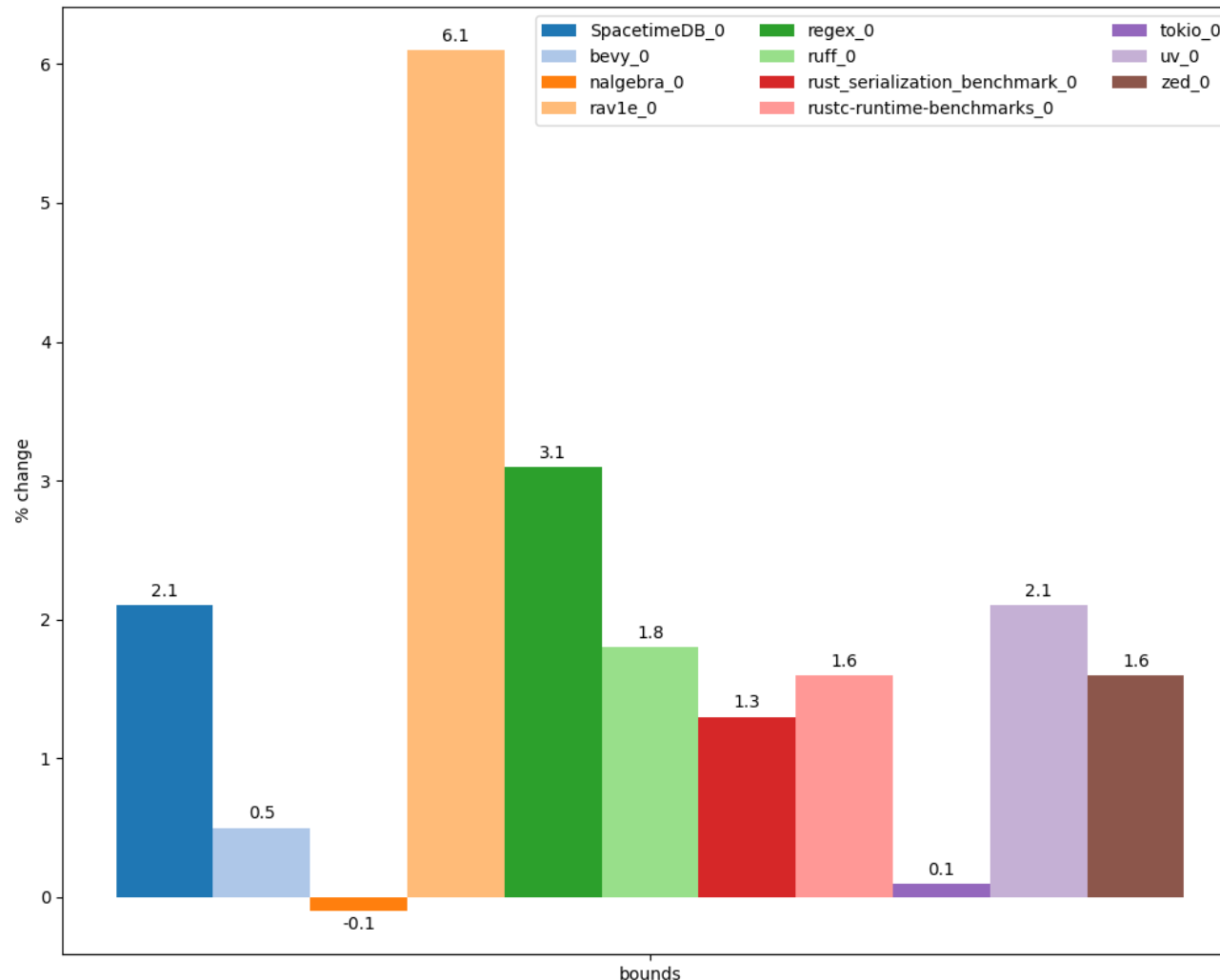
```
let table: [u64; LOOKUP_TABLE_SIZE] = ...  
... table[table & (LOOKUP_TABLE_SIZE - 1)] ...
```

- AND is *usually* faster (e.g. can run on more ALUs in processor)
- Unfortunately optimization (currently) applicable only to arrays

Prevalence

- 20% of checks in Rust compiler are bounds checks
 - Based on [analysis of binary code](#)
- In inner loops checks 80% contain only bounds checks
 - These checks prevent vectorization of surrounding loops
 - Data collected with [LLVM plugin](#) based on rustc and oxipng analysis

Performance improvement after disabling bounds checks



We disabled instrumentation in user code and checks in stdlib (containers, slices, strings, etc.)

Hardened C/C++

- C/C++ provides limited support for memory safety:
 - StackProtector
 - `_FORTIFY_SOURCE`
 - Hardened STL (C++ only)
 - `-fsanitize=bounds`, `-fsanitize=object-size`
- Complete, Rust-like correctness checks are impossible due to lack of information about lengths for raw pointers
 - E.g. only 20% of non-trivial memory accesses in Clang code can be checked
 - C++ Safe Buffers may solve this but requires non-trivial code modifications
- More information:
 - Hardening: current status and trends (C++ Zero Cost 2026)



Rust vs. Hardened C++

```
pub fn foo(x: &[i32], i: usize) -> i32 {  
    x[i]  
}
```

rustc -O -Cpanic=abort

<https://godbolt.org/z/jGT6adEYM>

```
foo:  
    cmp     rdx, rsi  
    jae     .LBB0_2  
    mov     eax, dword ptr [rdi + 4*rdx]  
    ret  
.LBB0_2:  
    push    rax  
    lea     rax, [rip + .Lanon.1]  
    ...  
    call    qword ptr [rip + panic_bounds_check]
```

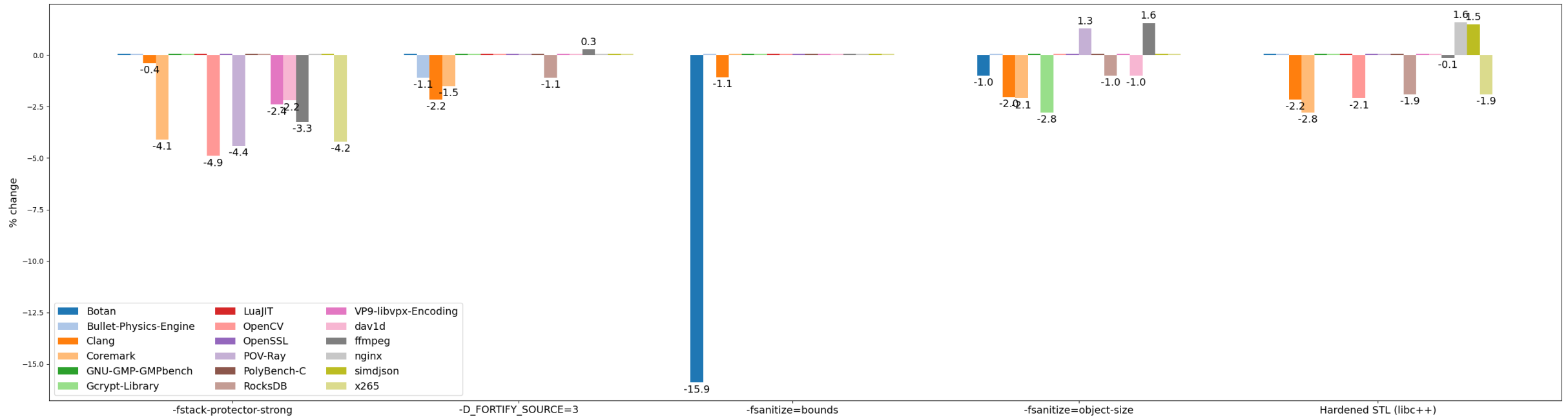
```
extern "C"  
int foo(std::span<int> s, size_t i) {  
    return s[i];  
}
```

gcc -O2 -D_GLIBCXX_ASSERTIONS

<https://godbolt.org/z/6MG5vdefK>

```
foo:  
.LFB821:  
    cmp     rdx, rsi  
    jnb     .L3  
    mov     eax, DWORD PTR [rdi+rdx*4]  
    ret  
.foo.cold:  
.L3:  
    push    rax  
    ...  
    call    std::__glibcxx_assert_fail
```

Overheads in Hardened C++



Unexpected performance improvements due to optimizer anomalies (bad vectorization, enabled inlining, etc.)

Conclusions

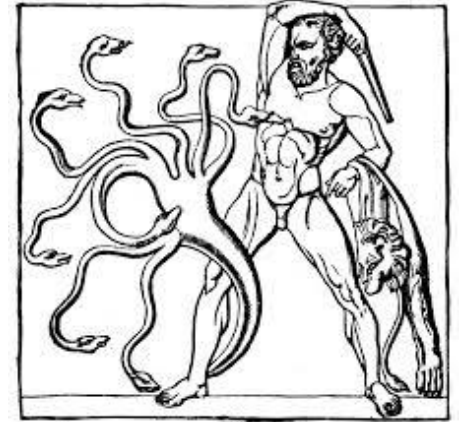
- Bounds checks in Rust are applied much more aggressively than in Hardened C/C++
 - Hardened C/C++ checks 5x accesses than Rust
- Overheads may reach 3-4% (and are comparable to Hardened C/C++)
 - But some tests may degrade by 10% and more (same applies to Hardened C/C++)
- There are many ways to avoid checks in hot code
 - Some of them won't be needed if delayed panics are finally implemented ([rust #50759](#))

Recommended readings

- [How to avoid bounds checks in Rust without unsafe](#) (Sergey Davidoff aka Shnatsel, Rust Secure WG lead)
 - “Bible” of bounds check elimination
- [Story-time: C++, bounds checking, performance, and compilers](#) (Chandler Carruth, SW foundations & PLs technical lead, creator of Carbon language)

Aliasing rules

Pointer aliasing



- Universal feature in programming languages
- Can two different references (pointers) address same object in program?
- Different languages make different choices:
 - No two pointers in program may refer to same data (e.g. Fortran 77)
 - Any two pointers may address same data (e.g. pre-C89)
 - Intermediate variants
 - E.g. strict (type-based) aliasing in C89 and restrict annotations in C99

Why aliasing is important?

- Unlimited aliasing is convenient for developer but reduces optimization opportunities for compiler
- Reduced aliasing enables many optimizations
 - E.g. vectorizers, CSE, GVN, LICM, DSE, etc.
 - Even used in CPU (for speculative execution of loads)
- Alias Analysis is one of the main components of any mature compiler
 - Identifies pointers that may (or may not) alias

```
int alias(int *a, int *b) {  
    *a = 0;  
    *b = 1;  
    return *a;  
}
```

```
int noalias(int * restrict a, int * restrict b) {  
    *a = 0;  
    *b = 1;  
    return *a;  
}
```

<https://godbolt.org/z/nqdax4vEP>

alias:

```
mov    DWORD PTR [rdi], 0  
mov    DWORD PTR [rsi], 1  
mov    eax, DWORD PTR [rdi]  
ret
```

noalias:

```
mov    DWORD PTR [rdi], 0  
xor     eax, eax  
mov    DWORD PTR [rsi], 1  
ret
```

Alias analysis theory

- Precise *intraprocedural* alias analysis is
 - NP-complete without dynamic allocations ([Pointer-induced aliasing](#), 1991)
 - Undecidable otherwise ([Undecidability of static analysis](#), 1992)
- Aliasing is very rare in real programs
 - 98% of pointers in SPEC benchmarks address only one object
 - [Dynamic Points-To Sets](#) (Mock et al., 2001)
- Existing alias analyses are conservative and overestimate aliasing 1.5-20x
 - [Dynamic Points-To Sets](#) (Mock et al., 2001)
 - [Speculative Alias Analysis for Executable Code](#) (Fernandez et al., 2002)
 - [ORAQL — Optimistic Responses to Alias Queries in LLVM](#) (Doerfert et al., 2023)

Aliasing in Rust

- Limitations imposed by Borrow checker simplify alias analysis:
 - Mutable reference may not alias any other (mutable or shared) reference
 - While it exists, mutable reference is the only pointer to object
 - Object addressed by shared reference may not be changed while reference exists
- In practice this achieves similar effect as restrict in C/C++

```
pub fn foo(a: &mut i32,  
          b: &mut i32) -> i32 {  
    *a = 0;  
    *b = 1;  
    *a  
}
```



```
define i32 @foo(  
    ptr noalias ... %a,  
    ptr noalias ... %b) {  
    store i32 0, ptr %a  
    store i32 1, ptr %b  
    ret i32 0  
}
```



```
foo:  
    mov     dword ptr [rdi], 0  
    mov     dword ptr [rsi], 1  
    xor     eax, eax  
    ret
```

<https://godbolt.org/z/6WoTzoo5v>

Limitations

- Information about aliasing is represented via `noalias` attribute in function arguments
 - References that appear *inside* functions are not marked and can alias from LLVM standpoint
- May be solved via [alias.scope metadata](#) in future ([#16515](#))
- Workaround: create dummy-functions specifically for `noalias` annotations
 - Example: [rustc](#)

```
pub fn foo(a: *mut i32,  
           b: *mut i32) -> i32  
{  
    let a = unsafe { &mut *a };  
    let b = unsafe { &mut *b };  
    *a = 1;  
    *b = 2;  
    *a  
}
```



```
foo:  
    mov     dword ptr [rdi], 1  
    mov     dword ptr [rsi], 2  
    mov     eax, dword ptr [rdi]  
    ret
```

[https://godbolt.org/z/
WjY9rxex7](https://godbolt.org/z/WjY9rxex7)

Limitations

- One case of above problem is aliasing of references to structure fields
- E.g. when passing two vectors to function compiler fails to understand that their buffers do not alias
- Workaround: pass raw slices to functions

```
pub fn foo(a: &mut Vec<i32>,
           b: &Vec<i32>) {
    let n = a.len();
    let b = &b[..n];
    for i in 0..n {
        a[i] = b[i] + 100;
    }
}
```

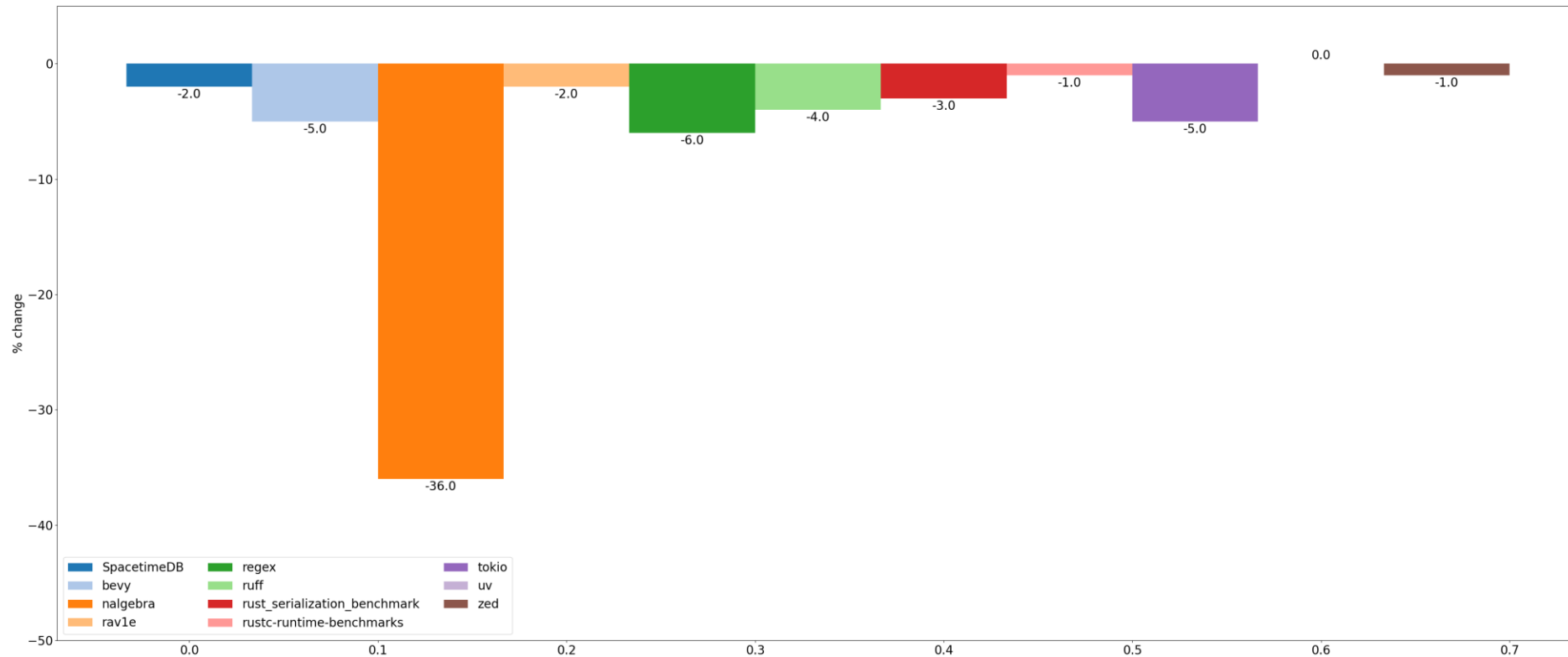


```
pub fn foo(a: &mut [i32],
           b: &[i32]) {
    let n = a.len();
    let b = &b[..n];
    for i in 0..n {
        a[i] = b[i] + 100;
    }
}
```

Limitations

- Aliasing rules apply only to references
- Raw pointers will still alias in Rust
 - This is worse than in C/C++ due to lack of strict aliasing rules

Efficiency of Rust aliasing rules



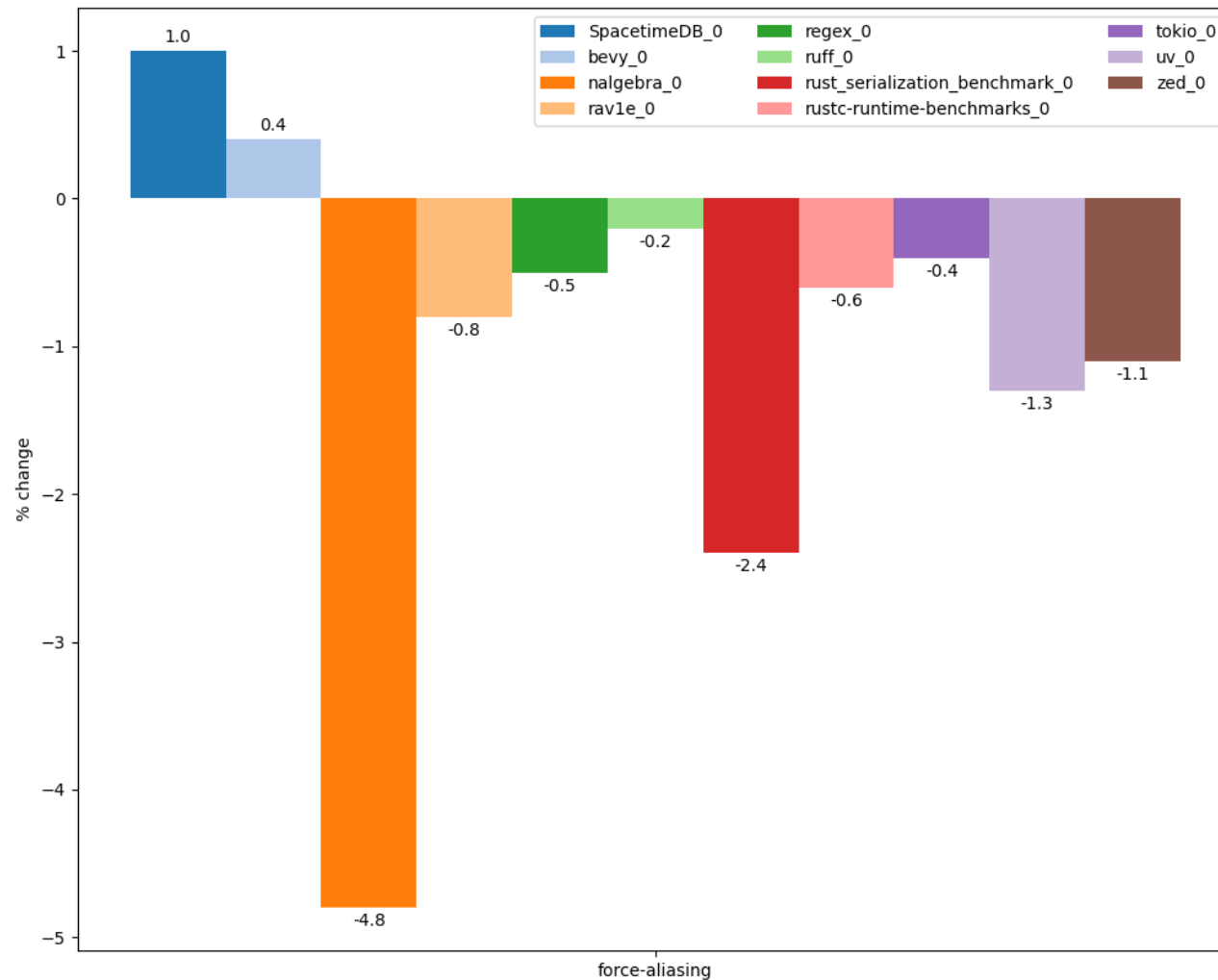
Precision of alias analysis – number of pointer pairs for which analysis could give definite answer (MustAlias / NoAlias)

Degradation of alias analysis precision in Rust after turning of reference rules

[Analysis methodology](#)

[Compiler branch](#)

Performance degradation after disabling aliasing rules



Situation in C/C++

- C and C++ disallow aliasing of pointers to different types
 - Except char *, signed/unsigned, scalar/vector, etc.
- But many projects ignore this rule via -fno-strict-aliasing flag
- C and C++ also allow developer to explicitly prohibit aliasing via restrict keyword
 - In practice this is seldomly used (Rust couldn't enable its aliasing rules for long time due to lots of hidden bugs in LLVM)



Conclusions

- Rust aliasing rules allow to
 - Increase analysis precision by 5% on average
 - Improve performance by 1-2% on average
- Can expect performance increase after further codegen improvements (aliasing metadata)

Recommended readings

- Complexity of alias analysis:
 - [Pointer-induced aliasing](#) (Landi, 1991)
 - [Undecidability of static analysis](#) (Landi, 1992)
- Aliasing in real code:
 - [Dynamic Points-To Sets](#) (Mock et al., 2001)
 - [Speculative Alias Analysis for Executable Code](#) (Fernandez et al., 2002)
 - [ORAQL — Optimistic Responses to Alias Queries in LLVM](#) (Doerfert et al., 2023)
- De-aliasing in CPU:
 - [Memory Disambiguation on Skylake](#)

Mandatory initialization

Mandatory initialization

- Uninitialized variables may cause information leaks and other errors
 - Password or address leaks (the latter compromises ASLR)
- Prevalence:
 - 10% CVE root cause in Microsoft products in 2018 (see [Killing Uninitialized Memory](#))
 - 12% exploitable bugs in Android (see [P2723](#))



Example

```
void foo() {  
    char password[32];  
    ...  
}  
void bar() {  
    char message[1024];  
    if (cond) strcpy(message, "...");  
    // Leaking password under !cond  
    printf(message);  
}  
void baz() {  
    foo();  
    bar();  
}
```

Rust approach

- Rust requires initialization of all variables
 - Local, global, heap
- Each variable must be assigned on all paths before first use

```
pub fn foo() {  
    let x = 10;  
    x  
}
```

OK

```
pub fn foo(cond: bool) -> i32 {  
    let x; // No mut needed  
    if cond {  
        x = 10;  
    } else {  
        x = 20;  
    }  
    x  
}
```

OK

```
pub fn foo(cond: u32) -> i32 {  
    let x; // No mut needed  
    if cond > 10 {  
        x = 10;  
    } else if cond <= 10 {  
        x = 20;  
    }  
    x  
}
```

NOT OK

Other languages

- Many other languages also require initialization (or autoinitialize with zeroes):
 - [C#](#), [Swift](#), [Go](#), [Java](#)
 - Hardened C/C++ (-ftrivial-auto-var-init), C++26 ([P2795](#))
 - Only local variables
 - Global variables and sbrk/mmap-buffers already zero-initialized
 - Heap can be zero-initialized via hardened allocator (e.g. hardened_malloc)
 - Enabled by default in Android user/kernel-space and Chrome

Issue with mandatory initializations

- Initializations are often redundant in practice
 - Will be overwritten with other values later without being read
- Compiler can't always eliminate such redundant initializations
- Example:

```
let mut file = File::open("example.bin");

while (...) {
    let mut buffer = [0u8; 1024]; // memset not removed

    let n = file.read_exact(&mut buffer)?;

    ...
}
```

Prevalence of redundant initializations

- How many initializations are redundant?
- Static estimate (via [LLVM plugin](#)):
 - Hardened Clang: +4% stores in loops (Clang benchmark)
- Dynamic estimate (via [Valgrind plugin](#)):
 - Compiling large C++ source file:
 - Clang: 11% unused writes
 - Hardened Clang (-ftrivial-auto-var-init): 23% unused writes (i.e. 2x increase)

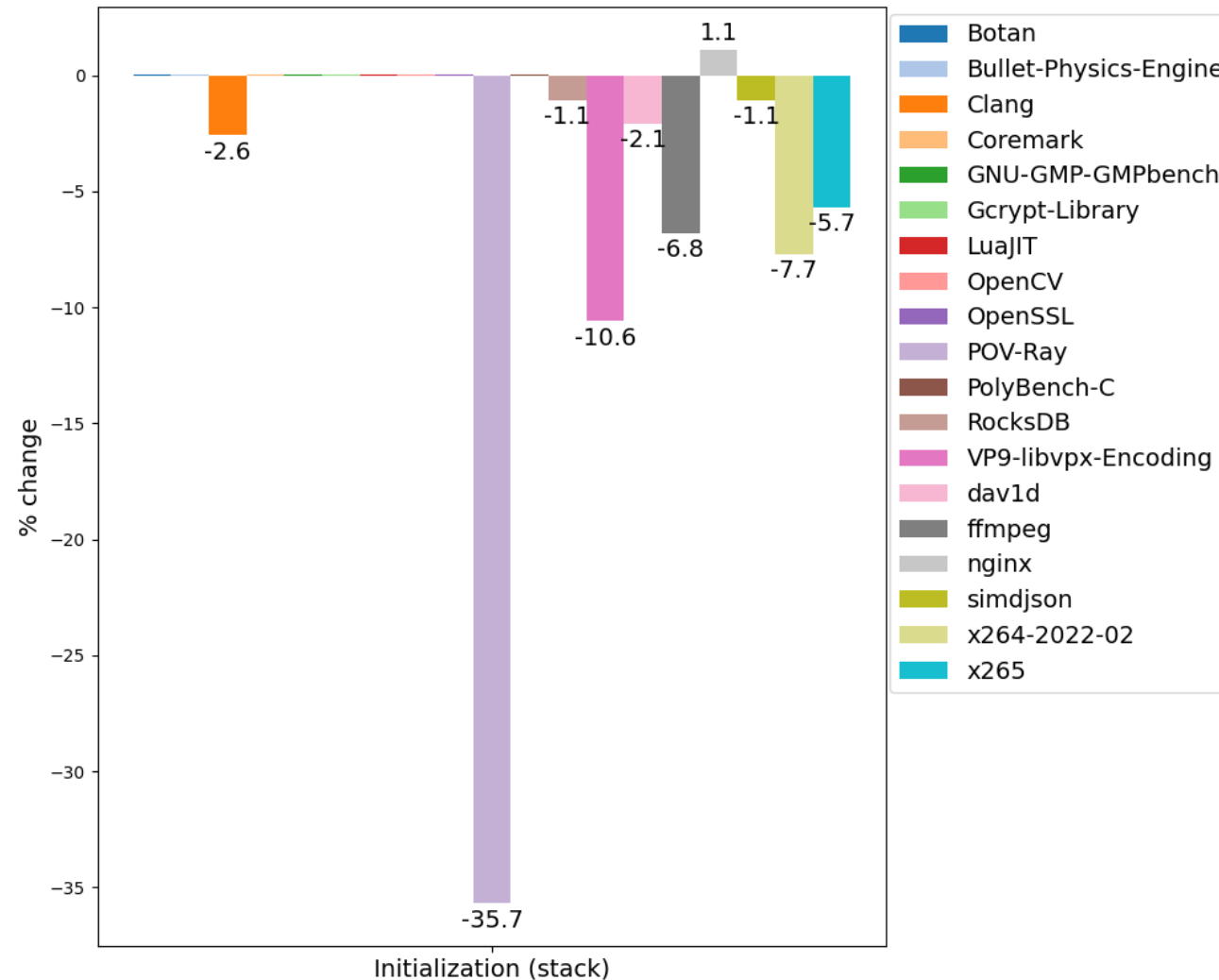
Overheads in benchmarks

- Unfortunately it's not possible to directly measure initialization overheads in Rust benchmarks
 - That would require extensive code rewrites
- We will illustrate overheads
 - Reported in various Rust projects
 - Measured for Hardened C/C++

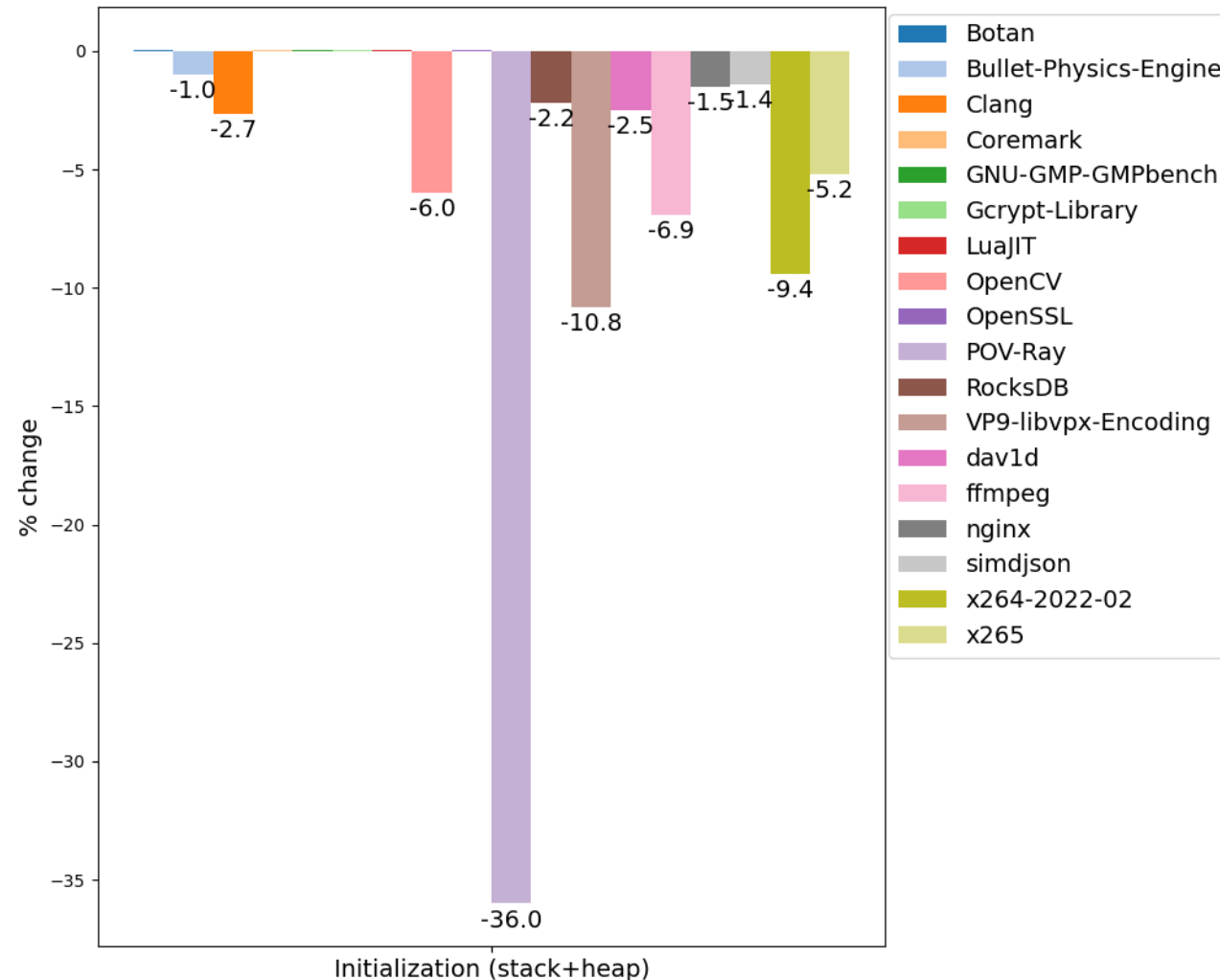
Example overheads in Rust programs

- What are the potential consequences for performance?
 - 7% in stdlib file IO ([rust #26950](#))
 - 20-30% in Hyper HTTP (see [tokio #1744](#))
 - 30% in Shadow simulator ([shadow #1643](#))
 - 25% in stdlib TcpStream ([RFC #837](#))
 - 1.5% in rav1d ([Making the rav1d Video Decoder 1% Faster](#))

Overheads with forced stack initialization for C++



Overheads with forced stack+heap initialization for C++



- We used crude approach to heap initialization (LD_PRELOAD + memset)
- More sophisticated implementation would have smaller overheads

Compiler optimizations

- Redundant scalar initializations can be optimized away by many SSA passes in LLVM (and are very cheap anyway)
 - DCE, BDCE, ADCE, InstCombine, etc.
- Redundant array initializations pose the main problem
 - DeadStoreElimination, MoveAutoInit, MemCpyOptimizer
 - To a lesser degree InstCombine

Workarounds

- Unfortunately APIs for uninitialized memory (MaybeUninit) are not very convenient
- One does not simply create a reference to uninitialized object and fill it:
“Creating a reference with `&/&mut` is **only allowed** if the pointer is properly aligned and **points to initialized data**” ([addr of mut](#))

Workarounds: Standard types

Arrays:

```
let buf :: [u8; 4] = unsafe {  
    let mut buf = MaybeUninit::<[u8;  
4]>::uninit();  
    let ptr = buf.as_mut_ptr();  
    for i in ... {  
        ptr.offset(i).write(...);  
    }  
    buf.assume_init()  
};
```

Structures:

```
let role :: Role = unsafe {  
    let mut uninit =  
MaybeUninit::<Role>::uninit();  
    let ptr = uninit.as_mut_ptr();  
    // Need write  
    // (instead of explicit assign)  
    // to avoid dropping  
    // uninitialized String  
    addr_of_mut!((*ptr).name)  
        .write("basic".to_string());  
    (*ptr).flag = 1;  
    (*ptr).disabled = false;  
    uninit.assume_init()  
};
```

Workarounds: Vectors

```
let mut buf = Vec<u8>::new();

let old_len = buf.len();
buf.reserve(data.len());

unsafe {
    let ptr = buf.spare_capacity_mut();
    for i in ... {
        ptr[idx].write(...);
    }
    buf.set_len(len + data.len());
}
```

Workarounds: IO

```
let mut buf = [MaybeUninit::uninit(); 4096];  
let mut buf = BorrowedBuf::from(&mut buf[..]);  
  
rd.read_buf(buf.unfilled())?;  
  
let data : &[u8] = buf.filled()  
  
// Work with buf.filled()
```

Conclusions

- Initialization overheads in Rust and Hardened C++ are on par
 - Higher than in C++26 due to heap initializations
- These overheads can be large in some scenarios (5-10% or more) and require manual code tuning:
 - Using dedicated APIs like MaybeUninit

Recommended readings

- [Killing Uninitialized Memory](#) ([video](#))
 - Report from Microsoft about enabling similar checks in Visual Studio
- [P2795: Erroneous behavior for uninitialized reads](#)
 - To be integrated to C++26

Symbol localization

Symbol localization

- Functions in translation unit (crate in Rust, cpp file in C++) can be used locally or globally (in other translation units)
 - Controlled with `static` keyword and anonymous namespaces in C++
 - Or with `pub`, `pub(crate)`, etc. in Rust
- Rust and C++ make opposing default choice:
 - In Rust functions are local for their module by default (top-level functions are local to their crate)
 - `pub(crate)`-functions are local to crate
 - Only `pub`-functions are visible across whole program

Localization in Rust

```
pub(crate) fn foo1(x: i32) {
    println!("Hello world {}", x);
}

mod mymod {
    pub(super) fn foo2(x: i32) {
        println!("Goodbye world {}", x);
    }

    pub fn foo3(x: i32) {
        println!("Nice world {}", x);
    }
}

pub use mymod::foo3;

pub fn bar() {
    foo1(1);
    mymod::foo2(2);
    mymod::foo3(3);
}
```

```
$ rustc +baseline test.rs -O --emit=asm --
crate-type=rlib -o- \
    | rustfilt \
    | grep globl
        .globl test::mymod::foo3
        .globl test::bar
```

Only foo3 and bar
are exported!

Compiler optimizations

- Local functions are much better optimized by compiler
- Usually such optimizations are related to API or ABI changes:
 - Interprocedural constant propagation (IPSCCP)
 - Replacing pass-by-reference with pass-by-value (ArgumentPromotion)
 - Removing dead arguments or return values (DeadArgumentElimination)
 - Relaxed calling convention (using coldcc in Transforms/IPO/GlobalOpt.cpp)
 - Interprocedural register allocation ([IPRA](#))
- And also
 - More aggressive inlining (e.g. local functions which are only called once are always inlined)
 - Better dead code elimination

Linker optimizations

- For shared libraries (DLLs) local functions
 - May be called directly (bypassing PLT/GOT)
 - Do not need to be resolved by loader at startup
- See more details in [Dynamic libraries and how to optimize them](#)

Example

```
#[inline(never)] // Simulate inliner fail
fn foo(b: bool, seed: i32, s: &[i32]) -> i32 {
    let mut ans = seed;
    for &x in s {
        ans += if b { 1 } else { ans ^ x }
    }
    ans
}
```

```
pub fn bar(n: i32, s: &[i32]) -> i32 {
    let mut ans = 0;
    for i in 0..n {
        ans += foo(true, i, s);
    }
    ans
}
```



foo:
lea eax, [rsi + rdi]
ret

Loop in foo was
optimized out

<https://godbolt.org/z/MM7Tchx49>

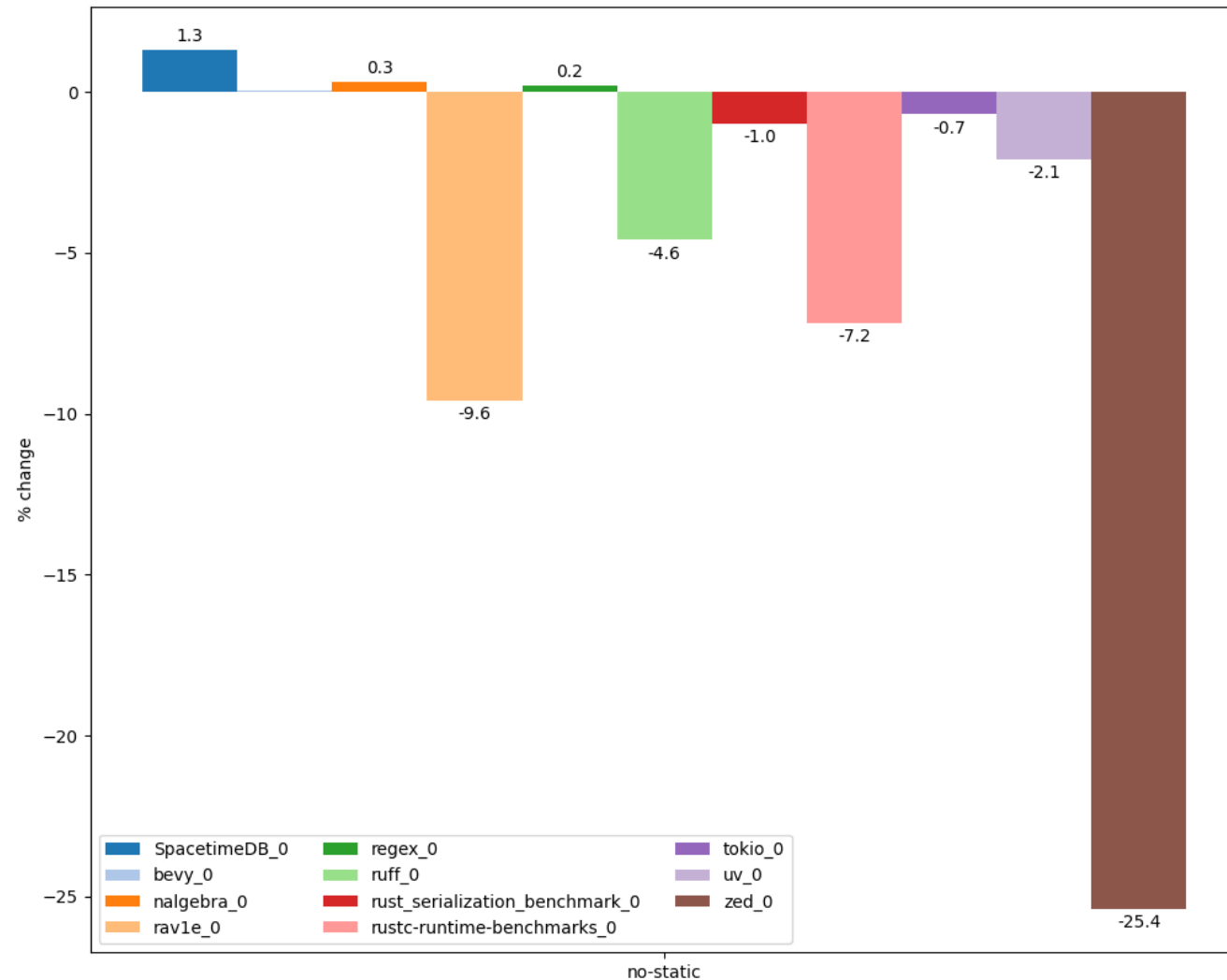
Comparison to C/C++

- Many authoritative coding guidelines suggest to localize functions:
 - [Google C++ Style Guide](#)
 - [LLVM Coding Standard](#) (Restrict Visibility)
 - [C++ Core Guidelines](#)
 - [Chromium C++ Style Guide](#)
- No way to localize private class methods in C++
 - [Prototype optimization](#)

Prevalence

- 80-90% of functions in Rust programs are local
 - Statistics collected via [this patch](#)
- In C/C++ results depend on the project:
 - 80-90% (clang, ffmpeg, git, opencv, bitcoin)
 - 40-50% (openssl, tmux, gcc, libuv)
 - (Statistics collected with [Localizer](#) tool)

Slowdown with disabled localization



Conclusions

- Function localization allows compiler to generate much more performant code (over 5%)
- Rust has more optimal performance-wise defaults

Recommended readings

- [Efficient C Tips #5 – Make ‘local’ functions ‘static’](#)

No inplace-initialization

Overview

- C++ has placement-new mechanism for constructing objects in arbitrary memory buffer:

```
void *addr = some_alloc(sizeof(A));  
A *ptr = new(addr) A(...); // Or std::construct_at
```

- Complemented by perfect forwarding:

```
std::vector<LargeStruct> vec;  
vec.emplace_back(large_struct_initializer);
```

- Rust lacks similar mechanism
 - Caused by mandatory object initialization

- Problem:

```
// Can compiler eliminate copy (from stack to heap)?  
let b = Box::new(LargeStruct { ... });
```

Why placement new is needed?

```
const SIZE: usize = 1024 * 1024 * 1024;

fn main() {
    let b: Box<[usize]> = if cfg!(fast) {
        vec![15; SIZE].into_boxed_slice()
    } else {
        // Problematic copy
        Box::new([15; SIZE])
    };
    black_box(&b);
}
```

```
# Can compiler eliminate copy?
$ LIMIT=$((8 * 1024))
$ rustc +baseline -O test.rs
$ (ulimit -s $ LIMIT; ./test)
thread 'main' has overflowed its stack
fatal runtime error: stack overflow

# Works with dedicated non-copying API
$ rustc +baseline --cfg fast -O test.rs
$ (ulimit -s $ LIMIT; ./test)
```

Why placement new is needed?

```
struct A {  
    std::array<int, 1024> vals = {};  
  
    A() = default;  
    A(const A &) = default;  
  
    A(int val) {  
        std::fill(vals.begin(), vals.end(), val);  
    }  
};
```

```
int main() {  
    std::vector<A> aa;  
    aa.reserve(NITER);  
  
    for (size_t i = 0; i < NITER; ++i) {  
#ifdef USE_PLACEMENT_NEW  
        aa.emplace_back(i);  
#else  
        A a(i);  
        aa.push_back(a);  
#endif  
  
        asm("" :: "r"(&aa) : "memory");  
    }  
  
    return 0;  
}
```

USE_PLACEMENT_NEW
~10% faster!

Compiler optimizations

- Move/copy elision
 - Detects and removes redundant moves and copies
- Can work more aggressively (compared to C++)
 - High-level IR (MIR)
 - Shallow move/copy (essentially move/copy in Rust is just a memcpy)
- Modern C++ compilers do not support complete copy elision optimization
 - Only limited optimizations like RVO, NRVO, etc.
 - Mainly due to lack of high-level IR (ClangIR to the rescue?)
 - See [Ultimate Copy Elision](#) work by Roman Rusyaev ([GitHub](#))
- Implementation in compiler:
 - MIR (high-level IR): CopyProp, DeadStoreElimination, DestinationPropagation, RenameReturnPlace
 - LLVM: MemCpyOptPass

Issues with move/copy elision

- Depends on compiler version
 - Not always monotonically
- Works only for release builds
 - Copies in debug builds are not optimized and may cause stack overflow or significant slowdown
- Language support for inplace-initialization is one of project goals in 2026:
 - [In-place initialization](#)

Workarounds

- Same as for mandatory initialization:
 - Use dedicated APIs to create uninitialized buffers and then initialize them:
 - E.g. `Box/Rc::new_uninit`, `Vec::with_capacity/set_len`
- Also other APIs e.g. `Vec::into_boxed_slice`

Conclusions

- Copy elision has more opportunities in Rust than in C++
- But can't fully compensate for lack of language support for inplace initialization
 - Like placement new and perfect forwarding
- Language developers are aware of this problem and try to solve it:
 - [Rust Project Goals 2026: In-place initialization](#)
 - [Rust Project Goals 2026: MIR move elimination](#)

Recommended readings

- [Ultimate Copy Elision](#)
 - Comprehensive overview of C++ copy elision by Roman Rusyaev

Structure optimizations

Alignment of structure fields

- Many languages (C, C++, C#, Swift) preserve order of fields in structures according to source code
- Fields need to be properly aligned by adding padding bytes in between
 - Alignment is necessary due to HW constraints
 - Unaligned memory access may cause exception or slowdown
- Alignment speeds up access to fields but reduces D\$ locality

```
struct RandomOrder {  
    uint8_t  m_byte1;  
    uint64_t m_long;  
    uint8_t  m_byte2;  
    uint32_t m_int;  
    uint8_t  m_byte3;  
};
```



Structures in Rust

- Rust ABI does not guarantee order of fields in structs
- This allows compiler to freely modify it
- Fields can be reordered to
 - Reduce D\$ pressure (by reducing amount of padding)
 - Optimize placement of niches (see below)
- In theory compiler could also co-allocate fields that are frequently used together
 - Could be enabled via PGO or static analysis (like -fipa-struct-reorg in GCC)
- And extract rarely used fields (structure peeling)

Layout optimizations



- Struct fields are sorted according to decreasing alignment to reduce struct size

```
pub struct Foo {  
    pub b: bool,  
    pub a: i64,  
    pub c: i16,  
} // 16 bytes total
```

```
pub fn fun(val: Foo) -> i64  
{  
    val.a  
    + (val.b as i64)  
    + (val.c as i64)  
}
```

```
fun:  
    movzx    ecx, byte ptr [rdi + 10]  
    add      rcx, qword ptr [rdi]  
    movsx    rax, word ptr [rdi + 8]  
    add      rax, rcx  
    ret
```

<https://godbolt.org/z/MxYh7bv34>

```
#[repr(C)]  
pub struct Foo {  
    pub b: bool,  
    pub a: i64,  
    pub c: i16,  
} // 24 bytes total
```

```
fun:  
    movzx    ecx, byte ptr [rdi]  
    add      rcx, qword ptr [rdi + 8]  
    movsx    rax, word ptr [rdi + 16]  
    add      rax, rcx  
    ret
```

<https://godbolt.org/z/GMoejn7rd>

C++ status

- Many static analyzers detect suboptimal field order:
 - Clang-tidy, PVS-studio, etc.
 - Pahole (uses debuginfo)
- clang-reorder-fields – LLVM-based tool for reordering fields to minimize padding
- In theory with Fat LTO compiler could reorder fields automatically
 - Already done in LCC compiler for Elbrus

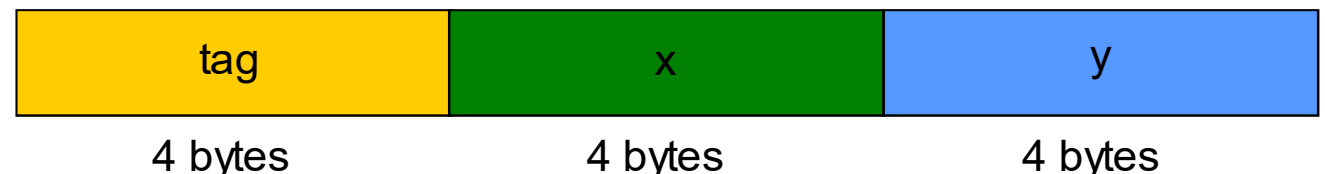
Internals of Rust enums

- Rust enums are analogous to `std::variant` in C++
- They may contain different types, identified by hidden tag field
- Enum object consists of
 - Tag (or discriminant) – index of actual type
 - This type's data

```
pub enum Foo {  
    A {x: i32, y: i32},  
    B {x: i32, y: i32},  
}  
  
pub fn fun(val: Foo) -> i32 {  
    match val {  
        Foo::A {x, ..} => x,  
        Foo::B {y, ..} => y,  
    }  
}
```

<https://godbolt.org/z/n8a1x9nWr>

```
fun:  
    mov     eax, dword ptr [rdi]  
    mov     eax, dword ptr [rdi + 4*rax + 4]  
    ret
```



Niches



- Some bit combinations may not correspond to any valid value of encoded type
- For example
 - `&x` or `NonZero<i32>` may not be zero
 - `bool` (8 bits) can only be `0b0` or `0b1`
 - Remaining 254 values are invalid
 - `char` (32 bits) may only contain values in `[0x0, 0xD7FF]` and `[0xE000, 0x10FFFF]` ranges
- Such invalid value ranges (not bits!) are called niches
- They may be used to keep tag of enum types (instead of using separate field)
 - So called Discriminant Elision

Examples of niches

- The most common (and guaranteed by the language) example: `Option<T>` with `T` being a NonNull type (e.g. `&U`)

$$\text{Option}\langle\&U\rangle = \begin{cases} \text{None} & = 0x0 \\ \text{Some}(\&u) & = \text{addr_of!}(u) \end{cases}$$

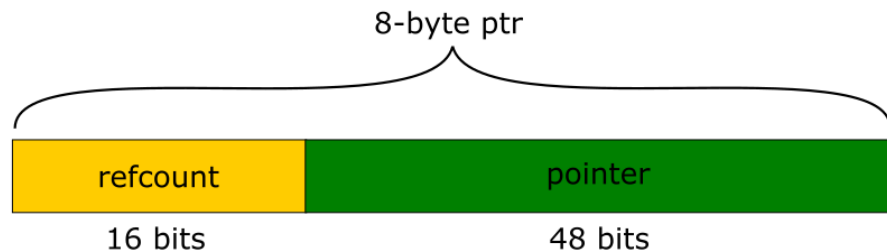
- Same niche may be used for multiple tags:
 - `size_of::<Option<Option<Option<bool>>>>() == 1`
- Similar optimization possible to lower bits of reference (or pointer)
 - Lower bits of aligned pointers are always 0
 - Can be used as niche
 - Currently only supported in nightly (-Zreference-niches, [rust #3204](#))

C++ status

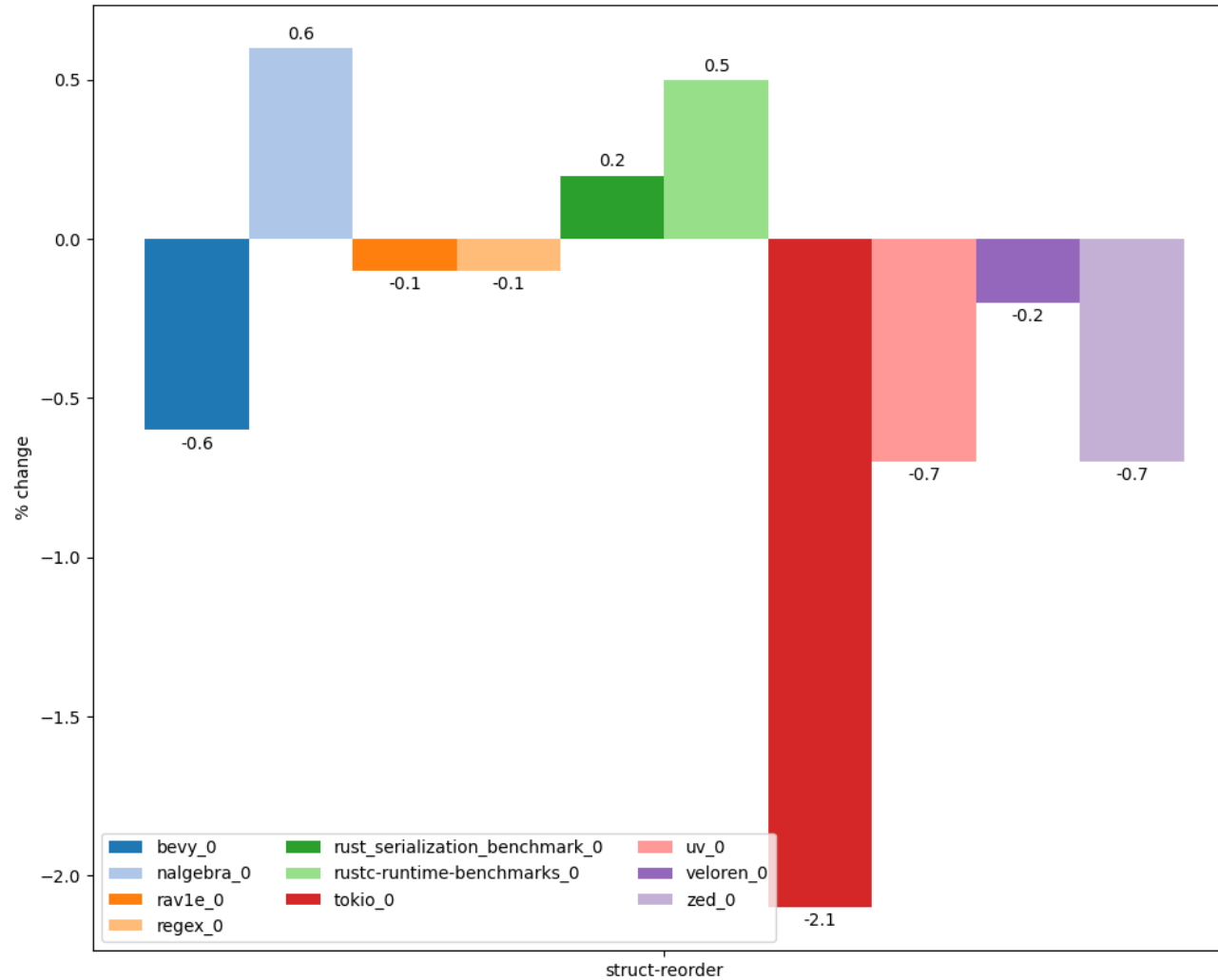
- `std::optional` and `std::variant` do not optimize for niches
 - Analog of `std::optional` with niched tags [can be implemented](#)
- `llvm::PointerIntPair`
 - Uses lower pointer bits to store int value
 - Similar to -Zreference-niches in Rust



- Lock-free implementations of atomic `shared_ptr` (e.g. in Folly) use free address bits for reference count
 - Allows for shorter and faster atomic operations



Overheads



- Disabled shouldMergeGEPs and store merging as they were too sensitive to field order

Conclusions

- Rust does not guarantee order of fields in structs
 - Allows more aggressive optimizations
 - And care in unsafe code
- Compiler may aggressively reorder fields and use them for tags
 - Analogous optimizations can be done manually in C++
- Field reordering in itself does not significantly affect performance but can enable or disable other important optimizations
 - Both in compiler and in processor
 - With both positive and negative performance effects

Recommended readings

- [The Lost Art of Structure Packing](#) (by ESR)

Arithmetic checks

Overview

- Arithmetic overflows may cause serious system crashes
 - ~1% CVE and 1.5% KEV in 2024
 - 23 place in [Mitre CWE Top 25 2024](#) (8 in [2019 rating](#))
- Well-known incidents:
 - Therac-25 radiation therapy machine (1985)
 - Ariane 5 catastrophe (1996)
- History (pre-Rust):
 - `-ftrapv` added in 2000 in GCC ([patch for -ftrapv option](#))
 - Wasn't well maintained and rotted quickly (e.g. [BZ #35412](#) opened 2008 and still not fixed)
 - John Regehr's works on IOC checker in [2010](#)
 - UBSan created in 2014
 - State-of-the-art solution
- Additional info:
 - [Hardening: current status and trends](#) (C++ Zero Cost 2026)



Example error (OpenSSH 3.3)

```
nresp = packet_get_int();  
if (nresp > 0) {  
    // Overflow to zero ...  
    response = xmalloc(nresp*sizeof(char*));  
    // ... and cause heap buffer overflow here  
    for (i = 0; i < nresp; i++)  
        response[i] = packet_get_string(NULL);  
}
```

Default arithmetic checks in Rust

- Some (not very important...) arithmetic checks enabled by default
 - Division by zero
 - Signed overflow on division (`INT_MIN / -1`)
 - Float-to-integer saturation
 - Can be disabled via `-Zsaturating-float-casts=off`
- Integer overflows in stdlib containers
 - Via `assert!`, `checked_add`, etc.

Important missing checks

- For historical reasons some important checks are missing
 - And can't even be enabled via flag
- E.g. overflow check on casting types via **as**
 - Need to use other APIs like **try_into/try_from**

```
fn main() {  
    0x100i32 as i8;  
}
```

```
$ ./test  
0
```

```
use std::convert::TryFrom;  
  
fn main() {  
    i8::try_from(0x100i32).unwrap();  
}
```

```
$ ./test
```

```
thread 'main' panicked at tmp57.rs:5:32:  
called `Result::unwrap()` on an `Err` value:  
TryFromIntError(())
```

Integer overflow checks in Rust

- By default integer overflows in user code are checked only in debug builds
- Two's complement semantics in release
- Can be enabled in release via `-C overflow-checks=on`
- Reason: too large overheads
 - “Plan is to turn on checking by default *if* we can get the performance hit low enough and also that it should leave room in the future to move towards universal overflow checking if it becomes feasible” ([Niko Matsakis](#), co-lead of Rust language design team)

Other overheads

- Overheads for overflow checks:
 - Compare + predictable conditional branch (1-2 ALU slots + I\$ pressure)
 - Hurt other optimizations (vectorizer, etc.)
- The first (less important) issue could potentially be fixed with hardware support

```
pub fn foo(xs: &[i32]) -> i32 {  
    let mut ans = 0;  
    for x in xs {  
        ans += x;  
    }  
    ans  
}
```

```
$ rustc ... test.rs https://godbolt.org/z/1x49jffv3  
...  
.LBB0_5:
```

```
// Loop successfully vectorized  
movdqu xmm2, xmmword ptr [rdi + 4*rax]  
padd    xmm0, xmm2
```

```
...  
$ rustc ... -Coverflow-checks=on test.rs  
...  
.LBB0_3:
```

```
// Loop not vectorized  
add     eax, dword ptr [rdi + rcx]  
jo      .LBB0_6  
add     rcx, 4  
cmp     rsi, rcx  
jne     .LBB0_3
```

<https://godbolt.org/z/43h1c87M5>

Defined overflow

- Unlike C/C++ integer overflows are not Undefined Behavior
 - Panic or Two's complement
- This deprives optimizer of some important information
- Can reduce precision of some analyses (especially ScalarEvolution) and disable some optimizations in LLVM
 - See [How undefined signed overflow enables optimizations in GCC](#)

Defined overflow: Example

From [Garbage In, Garbage Out](#) (Chandler Carruth)

```
for (;;) {  
    int c1 = buf[i1];  
    int c2 = buf[i2];  
    if (c1 != c2)  
        return c1 - c2;  
    i1++;  
    i2++;  
}
```



```
.LBB0_1:  
    movzx    eax, byte ptr [rdx + rsi]  
    movzx    edi, byte ptr [rdx + rcx]  
    inc      rdx  
    cmp      al, dil  
    je       .LBB0_1
```

<https://godbolt.org/z/Yssq68Gja>

```
loop {  
    let c1 = unsafe {  
        *buf.get_unchecked(i1 as usize)  
    } as i32;  
    let c2 = unsafe {  
        *buf.get_unchecked(i2 as usize)  
    } as i32;  
    if c1 != c2 {  
        return c1 - c2;  
    }  
    i1 += 1;  
    i2 += 1;  
}
```



Compiler has
to assume
potential
overflow

```
.LBB0_2:  
    movsxd   rdi, edi  
    movzx    eax, byte ptr [rdx + rdi]  
    movsxd   rsi, esi  
    movzx    ecx, byte ptr [rdx + rsi]  
    inc      edi  
    inc      esi  
    cmp      al, c1  
    je       .LBB0_2
```

<https://godbolt.org/z/c1sMcxfSP>

Problem with inclusive ranges

- Loops with included upper bound are less optimized by compiler:

```
for i in 1..n {  
    total += i;  
}
```

- Due to potential overflow (when `n==T::MAX`) compiler has to perform additional check:

```
while i <= n {  
    total += i;  
    if i >= n {  
        break;  
    }  
    i += 1;  
}
```

- Developers can help compiler by converting to exclusive loop:

```
for i in 1..(n+1)
```

- Or using [internal iteration](#) (`.for_each`, `.fold`, etc.)

Compiler optimizations

- Integer overflows are not well optimized by compiler
 - E.g. Presburger arithmetic is decidable but its complexity is $O(2^{2^N})$
- Only ~30% of arithmetic checks are eliminated when building Rust compiler
 - Based on [analysis of binary code](#)
- This is the main reason for lack of arithmetic checks in release builds

Workarounds

- Unsafe APIs:

- unchecked_add, wrapping_add, checked_add, etc.
- to_int_unchecked
- std::hint::unreachable_unchecked

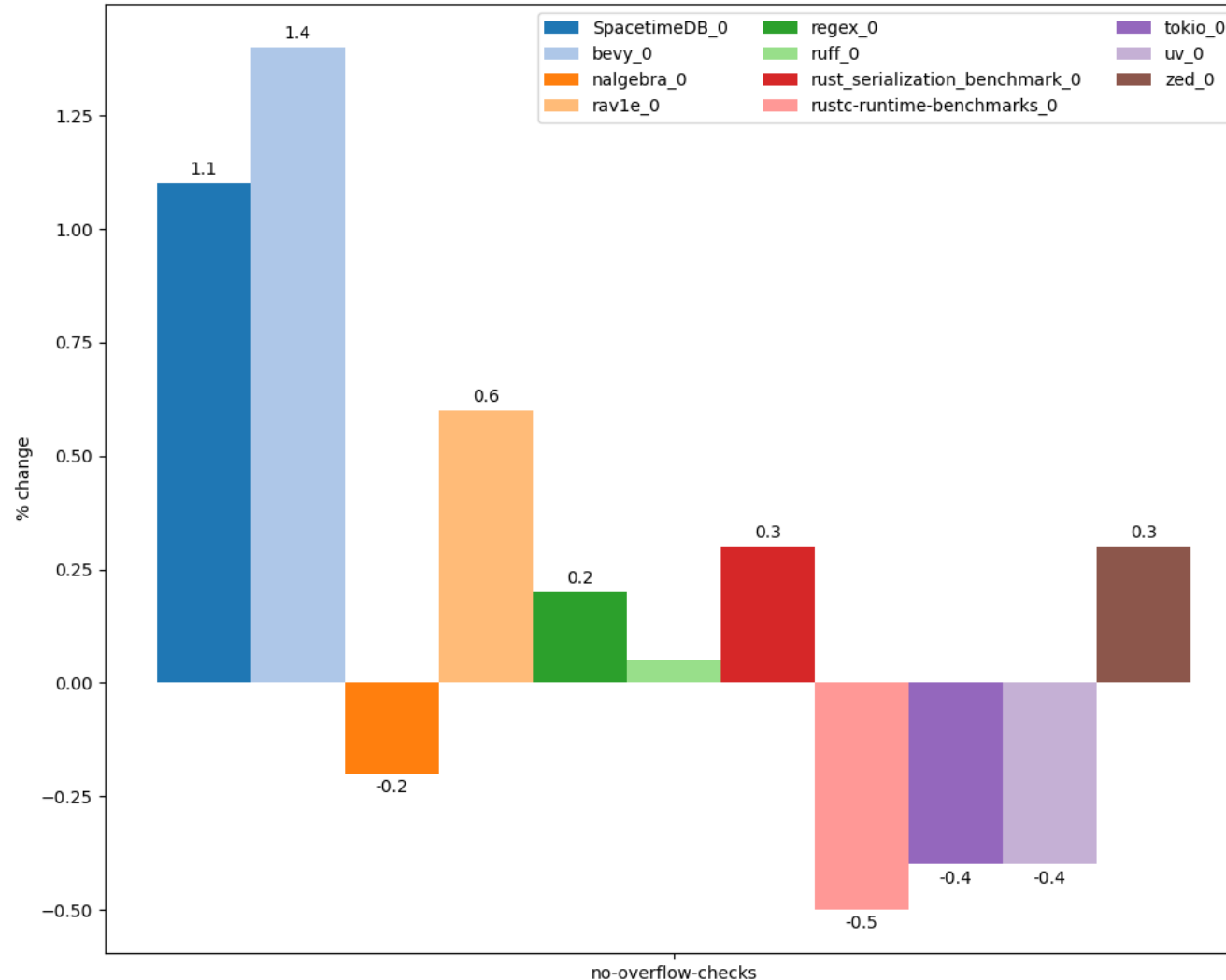
```
if x <= i32::MIN >> 1 || x >= i32::MAX >> 1 {  
    unsafe { std::hint::unreachable_unchecked() }  
}  
x * 2 / 2
```

- (checks in stdlib *can not be disabled*)

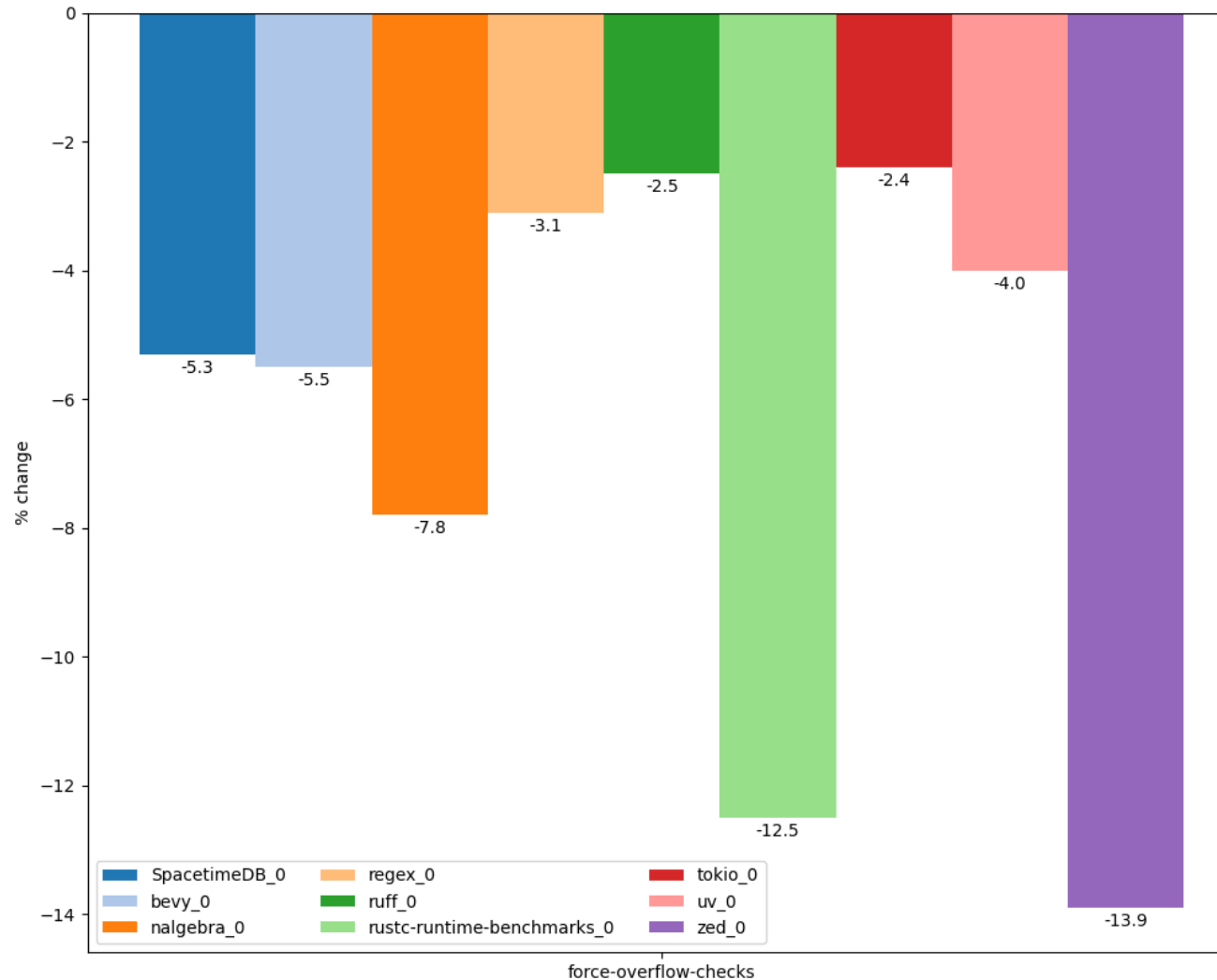
- Special stdlib types:

- NonZeroU32 to disable divide-by-zero checks
- Wrapping<T> for overflows

Improvements without default checks (division by zero, etc.)

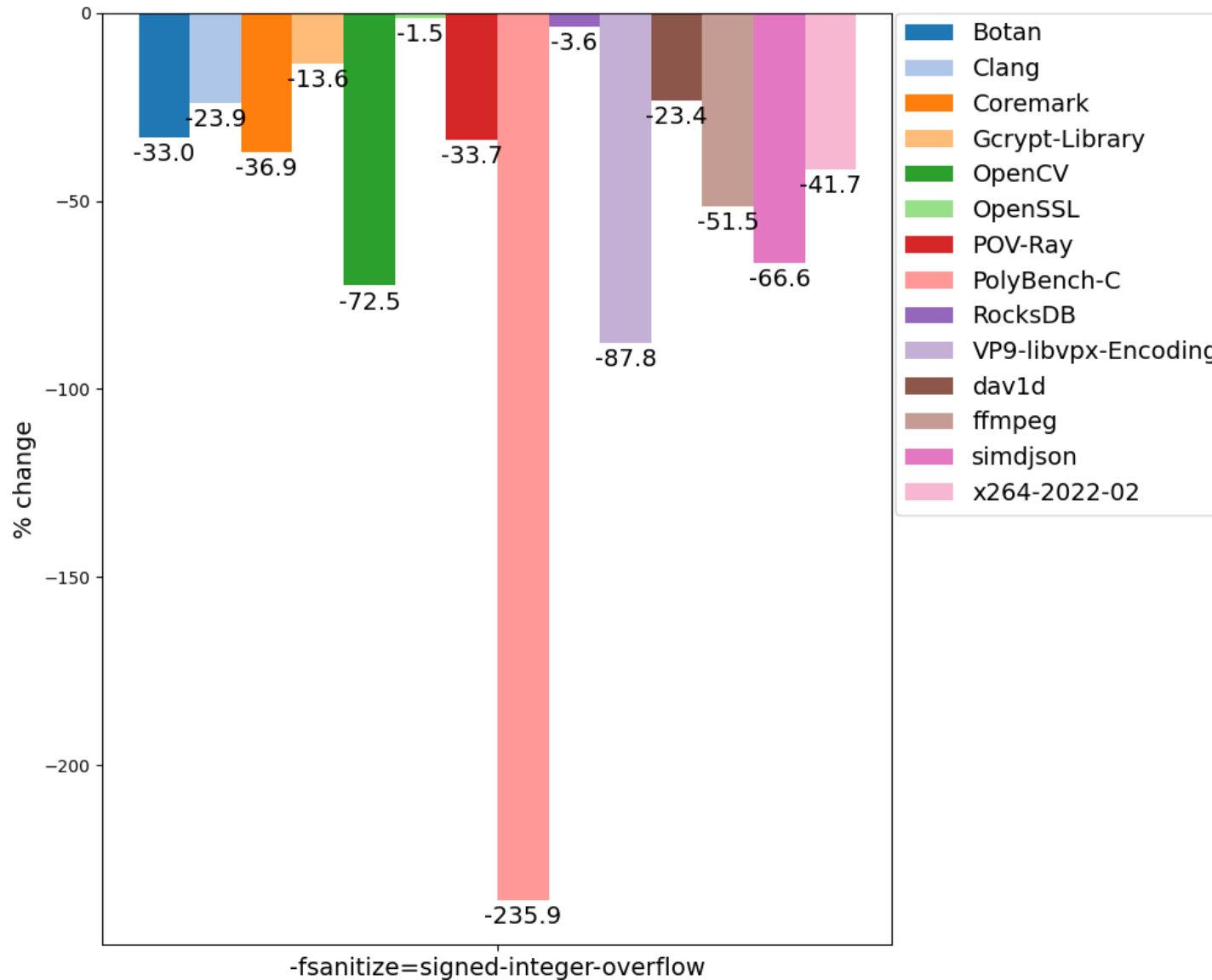


Slowdown with integer overflow checks



- Some tests missing due to crashes
- +30% checks in rustc

Slowdown in C++



- Some tests missing due to crashes
- Even larger overheads in C/C++ with UBSan

Conclusions

- Integer overflow is not UB in Rust
 - Prevents compiler from performing some aggressive optimizations
- Rust makes non-obvious choice about default arithmetic checks
 - Some really important errors like lossy casts are not checked but zero divisions are
 - Rare but may cause slowdown of ~1%
- Integer overflow checks have very large overhead (up to 10%)
 - Same as UBSan
 - Unlikely to be enabled by default in future

Recommended readings

- [RFC 560: Integer Overflow](#) (also [discussion](#) and [overview](#))
 - Attempt to enable integer overflow checks in release
- [Understanding Integer Overflow in C/C++](#)
 - Paper by John Regehr
- [How undefined signed overflow enables optimizations in GCC](#)
 - Paper about performance benefits of Integer Overflow UB in C/C++
- [Exploiting Undefined Behavior in C/C++ Programs for Optimization: A Study on the Performance Impact](#)
 - Paper about benefits of UB, including signed overflow, with benchmarks

Standard library

Overview

- Rust stdlib has more mandatory checks and in some cases uses more robust (and slow) algorithms compared to STL
 - Cargo build system greatly simplifies experiments with alternate implementations though
- On the other hand in other cases it can use faster container and algorithm implementations (compared to STL)

Standard library checks

- Stdlib contains a lot of mandatory checks (assert!, checked_add, etc.)
- Main types of checks:
 - Correctness of indices in containers (bounds checks, already considered above)
 - Size overflow checks in containers (capacity overflow)
 - Successful allocations in containers
 - Correctness of indices in UTF-8 container accesses (&str and String)
 - API-specific checks (comparator axioms, etc.)
- Similar checks are available in STL
 - Under `_GLIBCXX_ASSERTIONS` and `_LIBCPP_HARDENING_MODE` (Hardened STL)

Example

```
pub fn foo(length: usize) -> Vec<i32> {  
    vec![-1; length]  
}
```



```
lea    r15, [4*rsi]  
xor     r13d, r13d  
mov     rax, rsi  
// Check that 4*length does not overflow  
shr     rax, 62  
jne     .LBB0_11  
// Check that 4*length <= isize::max  
movabs  rax, 9223372036854775804  
cmp     r15, rax  
ja      .LBB0_11  
...  
// Check that alloc is successful  
call    qword ptr [rip + rustc::rust_alloc]  
test    rax, rax  
je      .LBB0_11  
...
```

<https://godbolt.org/z/oa67f1j5z>

Example

```
use std::io;

fn main() {
    let mut input = String::new();
    io::stdin()
        .read_line(&mut input)
        .unwrap();
    println!("{}", input);
}
```

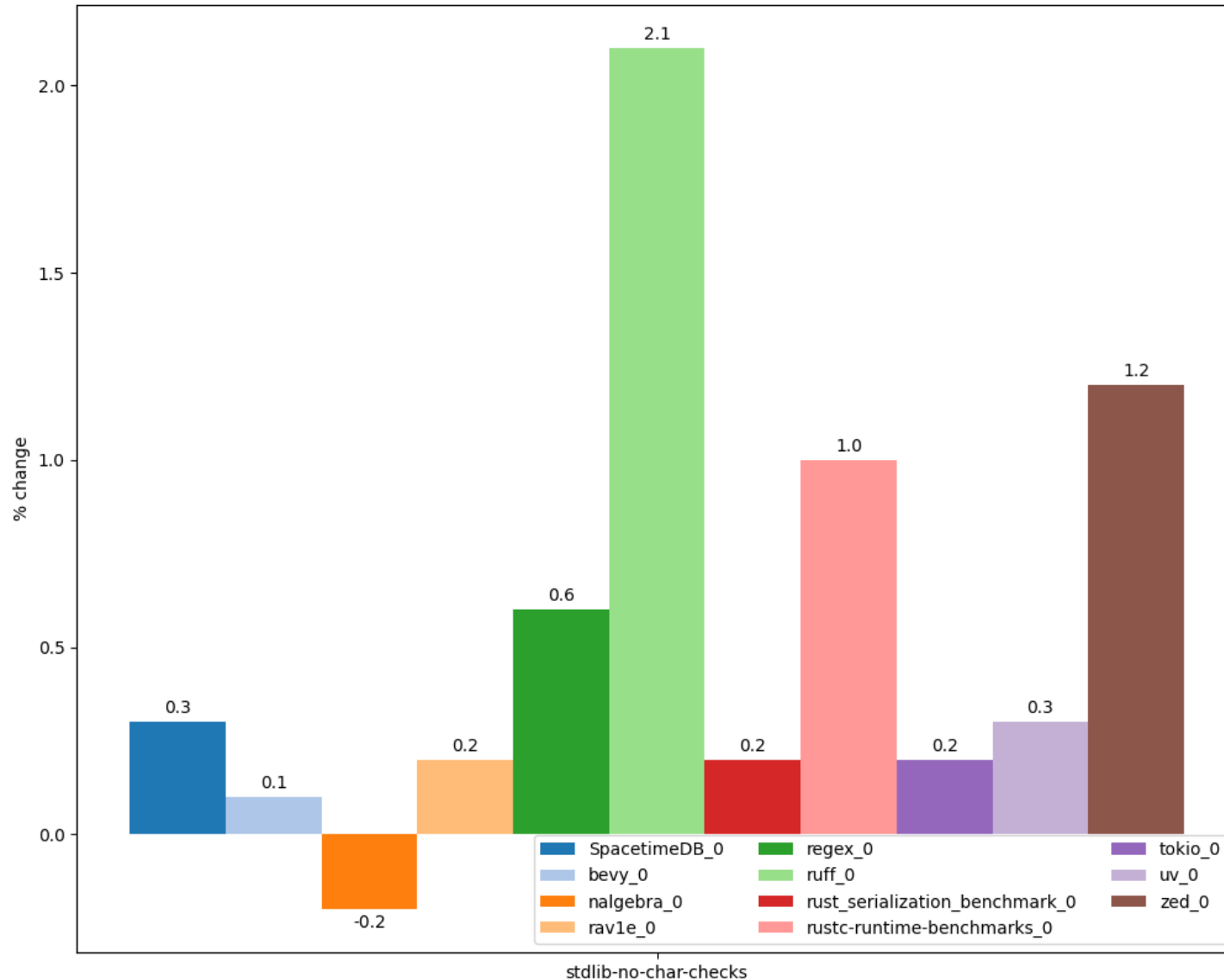
```
$ echo Привет | ./test
Привет
```

```
$ echo Привет | iconv -f UTF-8 -t KOI8-RU | ./test
thread 'main' panicked at test.rs:5:39:
called `Result::unwrap()` on an `Err` value:
Error { kind: InvalidData, message: "stream did not contain valid UTF-8" }
```

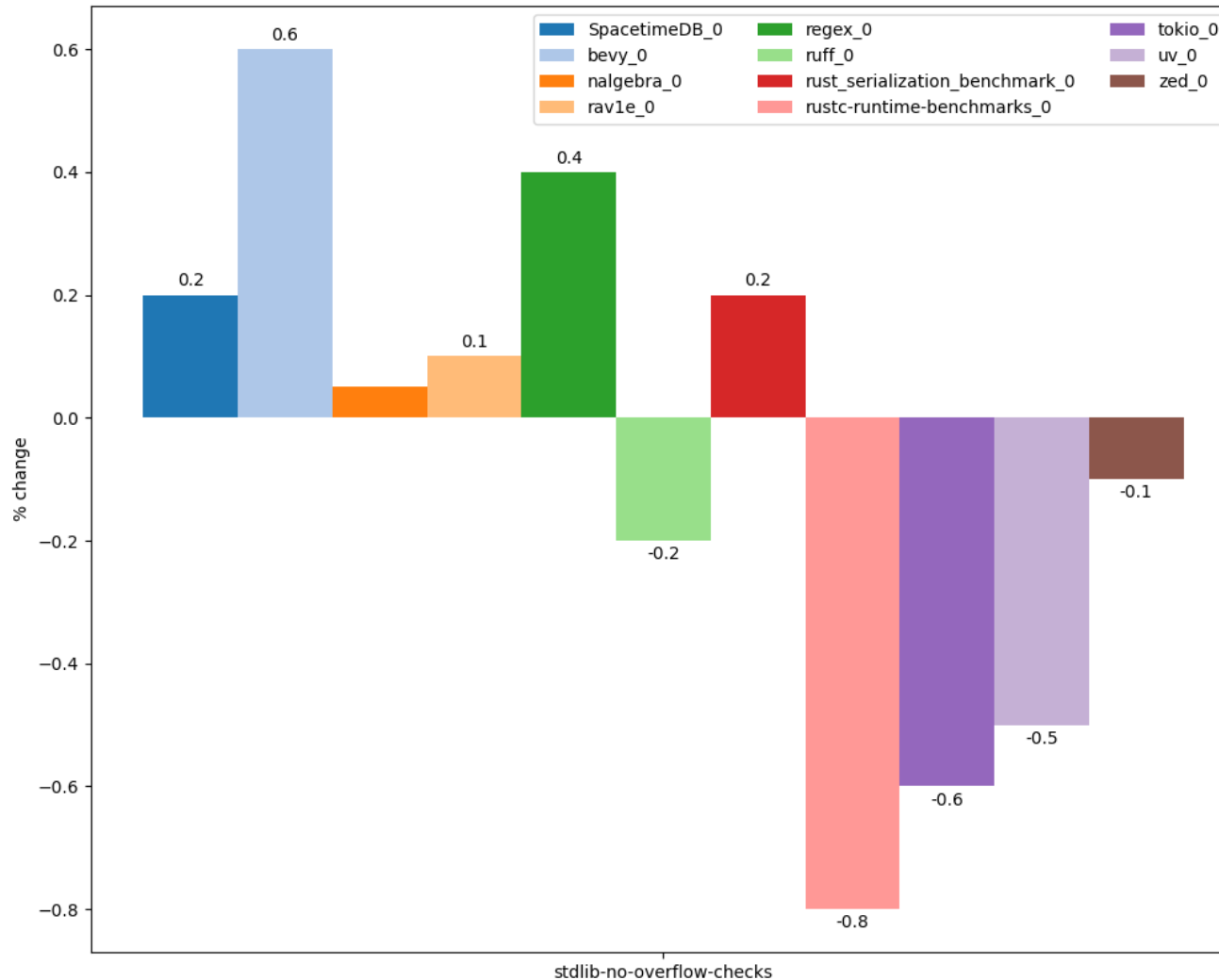
Other overheads

- Default hashing algorithm in hashtables (SipHash) is resilient against DoS attacks but is slower than analogs in STL
 - 2x slowdown against fxhash in [simple microbenchmark](#)
- Rust IO is unbuffered by default
 - Without BufReader programs may be 10x slower
- `io::stdout()` is line-buffered for non-TTY output ([rust #60673](#))
 - This may significantly slow down command-line tools (for cases like “./prog > file”)
- `print!/println!` macros use locked access to stdout for stable output from parallel threads

Improvements with disabled UTF-8 checks



Improvements with disabled container overflow checks



Optimizations in standard library

- Hashtables:
 - No requirement for element reference validity after modification (as in `std::unordered_map`)
 - Allows to use open-addressing instead of chaining (fewer allocations, cache-friendly)
- Search trees use B-tree (instead of RB-tree as `std::map`)
 - Fewer allocations, cache-friendly
 - 4x speedup on [simple microbenchmark](#)
- Modern ILP-friendly sorting algorithms (ipnsort, driftsort)
 - Modern STLs are implemented to [simplify vectorization](#) when sorting primitive types
 - 65% speedup on [simple microbenchmark](#)

Conclusions

- Rust stdlib includes both factors which negatively affect performance and more efficient (over STL) algorithms and containers
- Stdlib issues are not critical due to
 - Availability of many alternative implementations
 - Ease of integration of external libraries in Cargo ecosystem

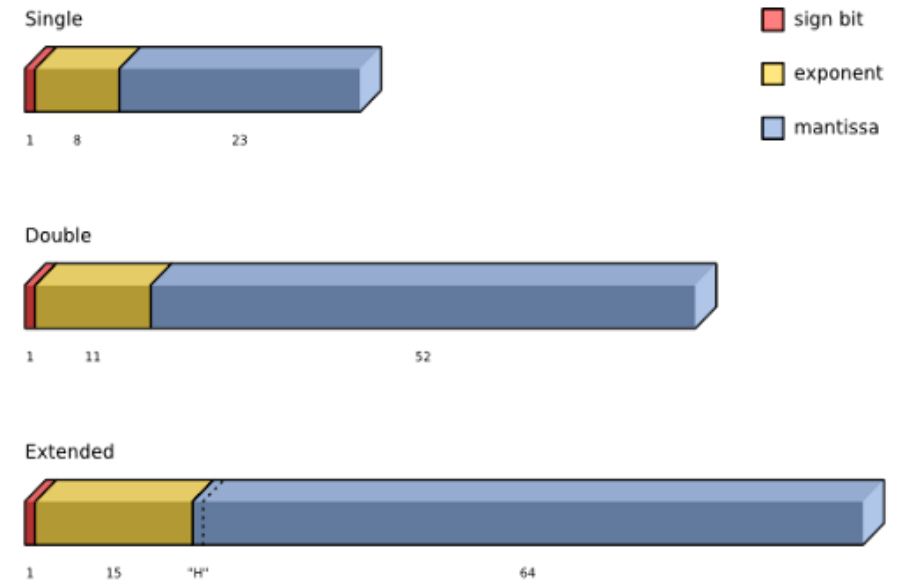
Recommended readings

- [Safety vs Performance. A case study of C, C++ and Rust sort implementations](#)

Lack of fast-math

Floating point computations

- IEEE 754 standard
 - Types and their binary representation
 - Operations on types
 - Rounding
- Most architectures and languages follow the standard (mostly)
- Reasons for discrepancies:
 - Architecture specifics
 - More opportunities for optimizations
- Fast math – umbrella term for optimizations that are enabled under relaxed IEEE 754 rules



Types of fast-math flags

- Algebraic:
 - Associativity
 - Unsignedness of zero
 - Replacing division with multiplication
 - Fusion of operations (e.g. FMA)
- Disallowing non-standard values:
 - NaN, Infinity

```
double foo(double val) {  
    return (val + 3.0) / 3.0;  
}
```

```
.LCPI0_0:  
    .quad    0x4008000000000000 # 3.0  
foo:  
    movsd    xmm1, qword ptr [rip +  
.LCPI0_0]  
    addsd    xmm0, xmm1  
    divsd    xmm0, xmm1  
    ret  
clang-21 -O2  
https://godbolt.org/z/Tqvsv4Tqs
```

```
.LCPI0_0:  
    .quad    0x4008000000000000 # 3.0  
.LCPI0_1:  
    .quad    0x3fd5555555555555 # 1/3  
foo:  
    addsd    xmm0, qword ptr [rip +  
.LCPI0_0]  
    mulsd    xmm0, qword ptr [rip +  
.LCPI0_1]  
    ret  
clang-21 -O2 -freciprocal-math  
https://godbolt.org/z/6sn7G4r4P
```

Fast math in other languages

- Different languages have different approach towards IEEE 754
- C Standard disallows fast-math optimizations (in Annex F)
 - Usually can be enabled with flags (e.g. -ffast-math)
 - Used e.g. in game engines
- Same applies for Swift language
- Java and C# allow to perform computations with higher precision than resulting type
- Go allows fusion of operations
 - Only if there are no explicit casts
- Fortran allows IEEE 754 violations to improve performance

Optimizations: Vectorization

- -fassociative-math and -fno-signed-zeros allow compiler to reorder loop iterations and vectorize loop

```
.LBB0_1:
  movss    xmm1, dword ptr [rdi + 4*rax]
  mulss    xmm1, dword ptr [rsi + 4*rax]
  addss    xmm0, xmm1
  # Loop NOT vectorized
  inc      rax
  cmp      rax, 256
  jne      .LBB0_1
```

clang-21 -O2 -fno-unroll-loops
<https://godbolt.org/z/v5vEb1Th>

```
#include <math.h>

float sum256(float *arr1,
             float *arr2) {
    float s;
    int i;
    s = 0.0f;
    for (i = 0; i < 256; i++) {
        s += arr1[i] * arr2[i];
    }
    return s;
}
```

```
.LBB0_1:
  movups   xmm1, xmmword ptr [rdi + 4*rax]
  movups   xmm2, xmmword ptr [rsi + 4*rax]
  mulps    xmm2, xmm1
  addps    xmm0, xmm2
  # Loop vectorized
  add      rax, 4
  cmp      rax, 256
  jne      .LBB0_1
```

clang-21 -O2 -fno-unroll-loops -fassociative-math -fno-signed-zeros
<https://godbolt.org/z/j6P1ohcTr>

Optimizations: Algebraic simplifications

- Fast-math may allow more simplifications or compile-time evaluation
- -fassociative-math + -fno-signed-zeros

<pre>double foo(double val) { return 3.0 + val - 3.0; }</pre>	<pre>foo: addsd xmm0, qword ptr [rip + .LCPI0_0] # +3.0 addsd xmm0, qword ptr [rip + .LCPI0_1] # -3.0 ret</pre> clang-21 -O2 https://godbolt.org/z/hjzWejv38	<pre>foo: ret</pre> clang-21 -O2 -fassociative-math -fno-signed-zeros https://godbolt.org/z/a5GME55MM
---	---	--

- -fassociative-math + -fno-signed-zeros + -freciprocal-math

<pre>double foo(double val) { return 3.0 * val / 3.0; }</pre>	<pre>foo: movsd xmm1, qword ptr [rip + .LCPI0_0] mulsd xmm0, xmm1 # * 3.0 divsd xmm0, xmm1 # / 3.0 ret</pre> clang-21 -O2 https://godbolt.org/z/nancn6j8x	<pre>foo: ret</pre> clang-21 -O2 -fassociative-math -fno-signed-zeros -freciprocal-math https://godbolt.org/z/dMeKc8PW9
---	---	--

Optimizations: Fused instructions

- Some architectures support fusion of floating point instructions
 - E.g. Fused Multiply Add in x86-64
- Rounding of these instructions may be different from sequence of roundings of their primitive operations
- Enabled via `-ffpcontract=on` flag

```
double foo(double *val) {  
    double acc = 1;  
    for (int i = 0; i < 100; ++i) {  
        acc = acc * val[i] + val[i + 1];  
    }  
    return acc;  
}
```

```
.LBB0_1:  
    mulsd    xmm0, xmm1 # multiply  
    movsd    xmm1, qword ptr [rdi + 8*rax]  
    addsd    xmm0, xmm1 # add  
    inc      rax  
    cmp      rax, 101  
    jne      .LBB0_1  
    ret
```

clang-21 -O2 -fno-unroll-loops
<https://godbolt.org/z/qTTY4r7Yx>

```
.LBB0_1:  
    vmovsd    xmm2, qword ptr [rdi + 8*rax]  
    vfmadd213sd    xmm0, xmm1, xmm2 # fused multiply-add  
    inc      rax  
    vmovapd    xmm1, xmm2  
    cmp      rax, 101  
    jne      .LBB0_1  
    ret
```

clang-21 -O2 -fno-unroll-loops -ffp-contract=on -march=haswell 145
<https://godbolt.org/z/zvPTo9fz4>

Optimizations: Finite only

- -ffinite-math-only enables optimizations that rely on lack of NaN or Inf values (and corresponding runtime exceptions)
- This allows to
 - Remove NaN and Inf checks
 - $x == x \Rightarrow \text{true}$
 - $x / x \Rightarrow 1$
- With -fno-signed-zeros:
 - $x - x \Rightarrow 0$
 - $0 / x \Rightarrow 0$
 - $x * 0 \Rightarrow 0$

```
int foo(double val) {  
    return val == val;  
}
```

```
foo:  
    ucomisd xmm0, xmm0  
    setnp    al  
    ret
```

clang-21 -O2

<https://godbolt.org/z/v5YEg7hbY>

```
foo:  
    mov     al, 1  
    ret
```

clang-21 -O2 -ffinite-math-only

<https://godbolt.org/z/TYab9sndr>

```
double foo(double val) {  
    return val * 0;  
}
```

```
foo:  
    xorpd   xmm1, xmm1  
    mulsd   xmm0, xmm1  
    ret
```

clang-21 -O2

<https://godbolt.org/z/3e8sM1dh9>

```
foo:  
    xorps   xmm0, xmm0  
    ret
```

clang-21 -O2 -ffinite-math-only -fno-signed-zeros

<https://godbolt.org/z/zoWrsfr1h>

Optimizations: Errno

- -fno-math-errno flag allows compiler to ignore errno global variable effects in math functions
- In particular this enables better inlining and simplification of library functions
- In Rust standard math operations are compiled to LLVM intrinsics which do not set errno

```
#include <math.h>
```

```
double foo(double x) {  
    return sqrt(x);  
}
```

```
foo:
```

```
xorpd    xmm1, xmm1  
ucomisd  xmm0, xmm1  
jb       sqrt@PLT  
sqrtsd   xmm0, xmm0  
ret
```

clang-21 -O2

```
foo:
```

```
sqrtsd   xmm0, xmm0  
ret
```

clang-21 -O2 -fno-math-errno

```
pub fn foo(num: f64) -> f64 {  
    num.sqrt()  
}
```

```
foo:
```

```
sqrtsd   xmm0, xmm0  
ret
```

Issues with fast-math: Checks

- Usage of Inf or NaN with -ffinite-math-only is UB
- Explicit checks for such values will be removed by compiler

```
#include <math.h>

int foo(float f) {
    return isnan(f);
}
```

```
foo:
    xor     eax, eax
    ucomiss xmm0, xmm0
    setp    al
    ret
```

clang-21 -O2
<https://godbolt.org/z/MPPoc5oea>

```
foo:
    xor     eax, eax
    ret
```

clang-21 -O2 -ffinite-math-only
<https://godbolt.org/z/nv33ffaqx>

Issues with fast-math: Rounding

- Some algorithms use special order of computations to reduce rounding error
- With -fassociative-math compiler may ignore this order and “optimize” these algorithms as if there is no rounding
- E.g. Kahan algorithm

```
float sum(float *arr) {  
    float s;  
    int i;  
    s = 0.0f;  
    for (i = 0; i < 256; i++) {  
        s = s + arr[i];  
    }  
    return s;  
}
```

```
sum_kahan:  
    xorps    xmm0, xmm0  
    xor      eax, eax  
.LBB0_1:  
    addss    xmm0, dword ptr [rdi +  
4*rax]  
    inc      rax  
    cmp      rax, 256  
    jne      .LBB0_1  
    ret
```

clang-21 -O2 -ffast-math -fno-unroll-loops -fno-vectorize
<https://godbolt.org/z/rYebWsfco>

```
float sum_kahan(float *arr) {  
    float s, c, t, y;  
    int i;  
    s = c = 0.0f;  
    for (i = 0; i < 256; i++) {  
        y = arr[i] - c;  
        t = s + y;  
        c = (t - s) - y;  
        s = t;  
    }  
    return s;  
}
```

Rust approach

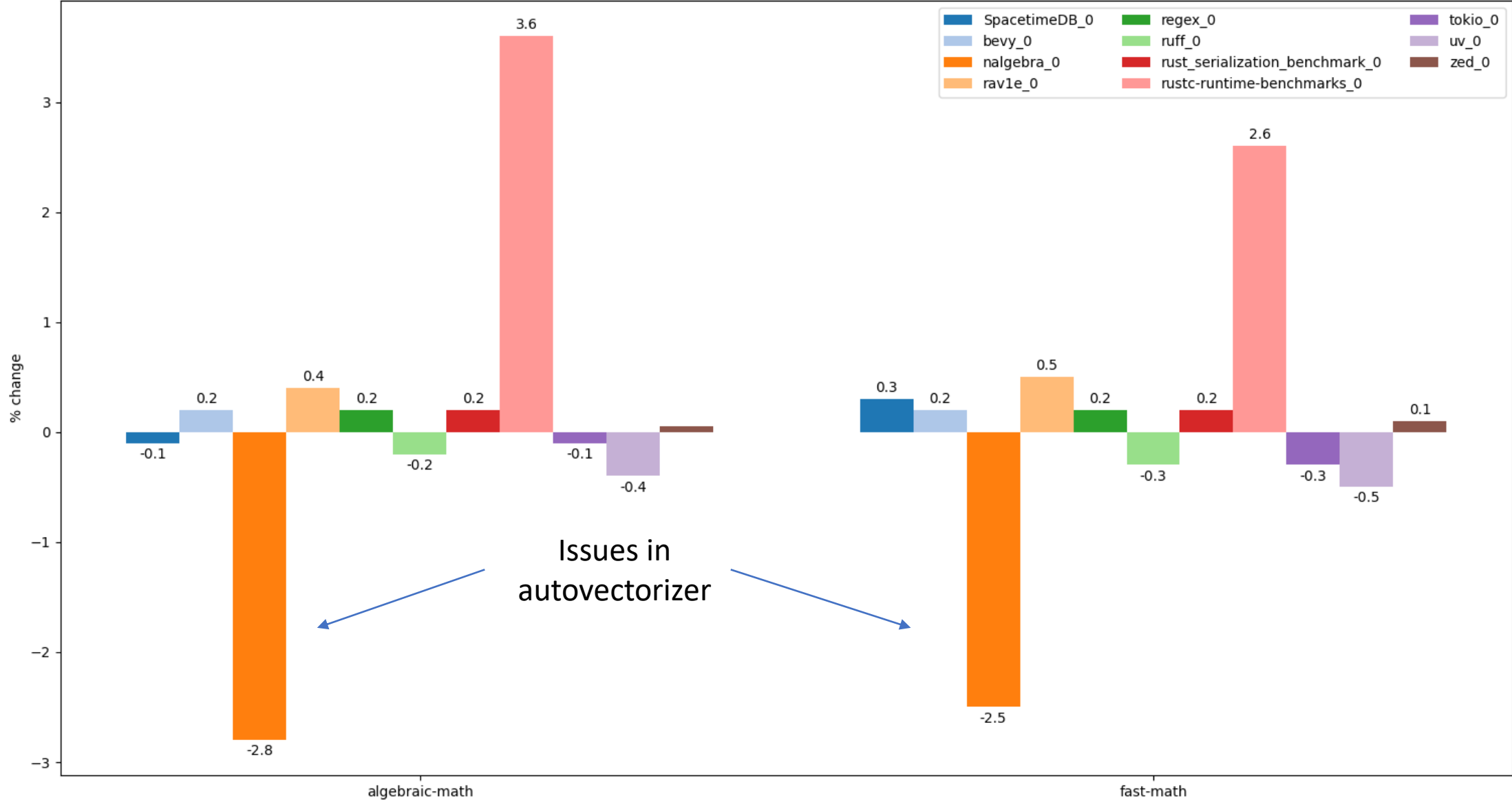
- Developers of Rust rejected to enable opportunities for global fast-math optimizations
- Due to potentially unsafe usage of these optimizations by users (especially in project dependencies)

“Everything about this flag fundamentally clashes with the idea of robust, compositional system design. It was a terrible mistake to ever add it to C compilers, and Rust should not repeat that mistake. Rust should instead explore alternative, less fragile ways of exposing those semantics.” (Ralf Jung, [rust #21690](#))

Workarounds: Intrinsics

- Rust provides dedicated intrinsics for enabling fast-math optimizations locally
 - `std::intrinsics::f{op}_fast`
 - `std::intrinsics::f{op}_algebraic`
 - `f(16/32/64)::algebraic_{op}`
- Disadvantages:
 - Currently require nightly (unstable) compiler build
 - Currently there are [no wrapper types](#) which hide usage of intrinsics
 - Authors of math libraries have to implement two versions of code (fast-math and normal)

Performance with fast-math



Issues in
autovectorizer

Conclusions

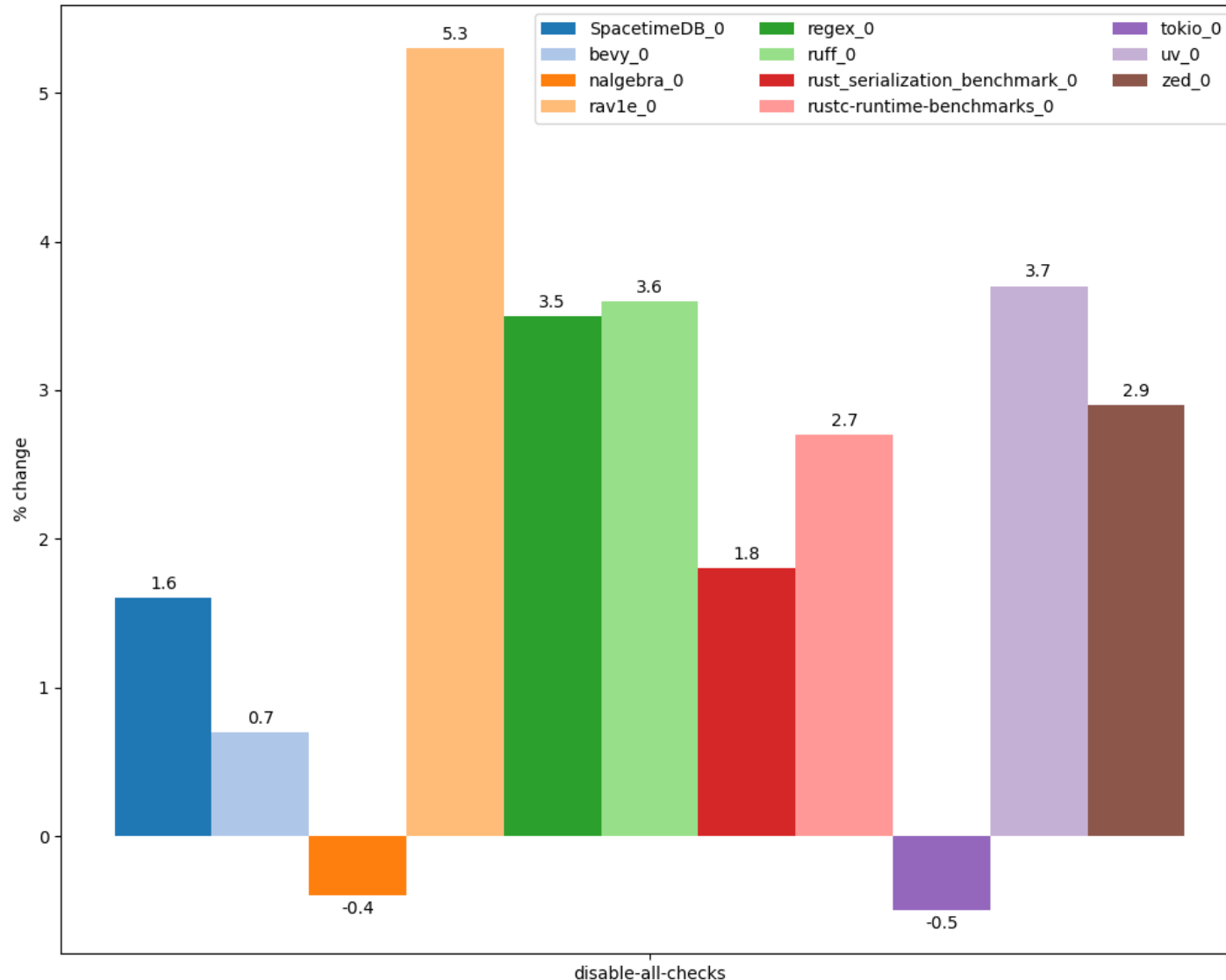
- Rust does not provide a way to enable fast-math optimizations globally
- Can be enabled for particular operations via intrinsics
 - Currently only in nightly toolchain
- Fast-math may both improve and degrade performance

Recommended readings

- [Optimizations enabled by -ffast-math](#) (Krister Walfridsson, GCC)
- [Towards Useful Fast-Math](#) (LLVM Dev Meeting 2024)
- [Beware of fast-math](#)
- [P3715R0: Tightening floating-point semantics for C++](#)
- [Taming Floating-Point Sums](#)

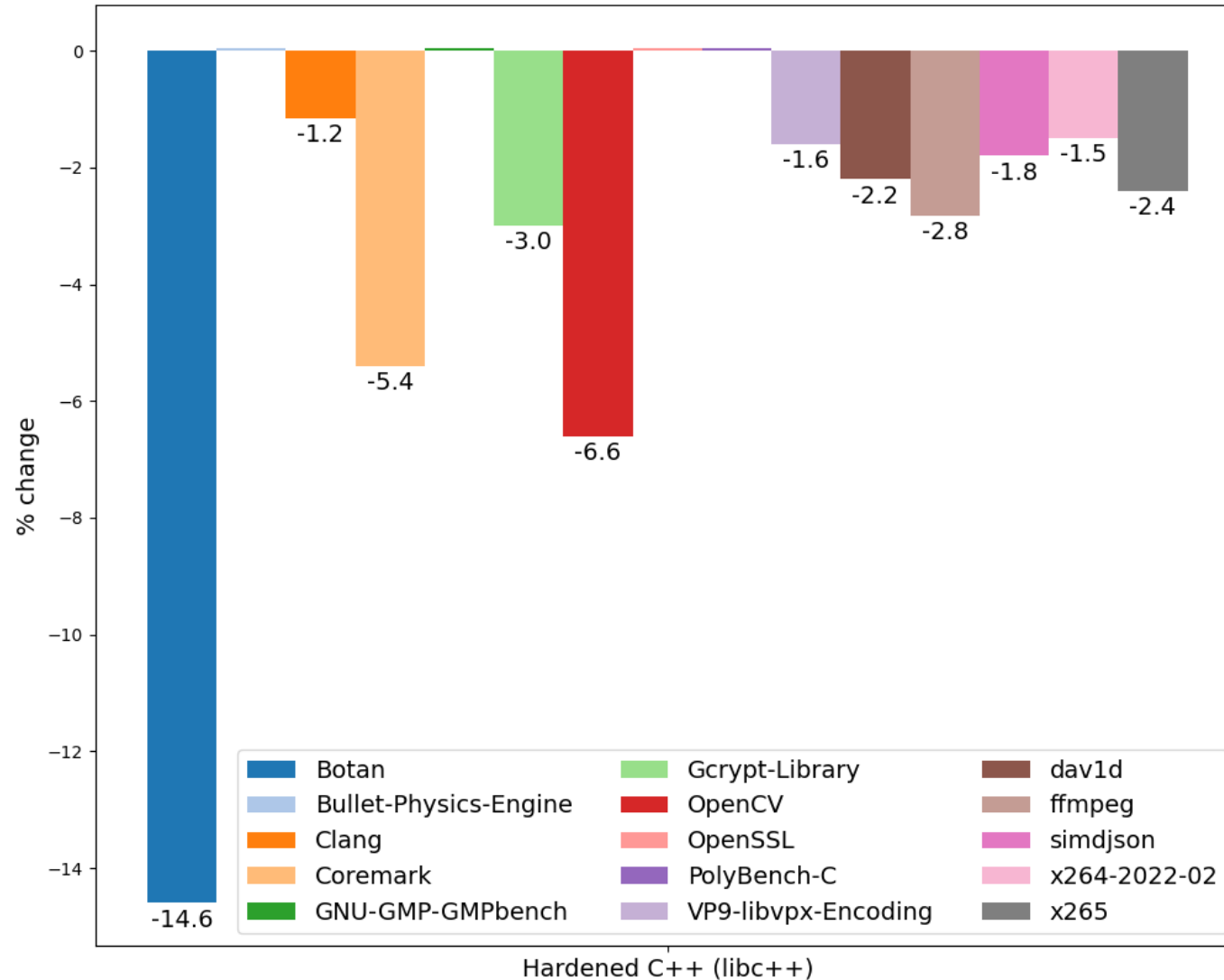
Conclusions

Speedup with disabling (most) Rust checks



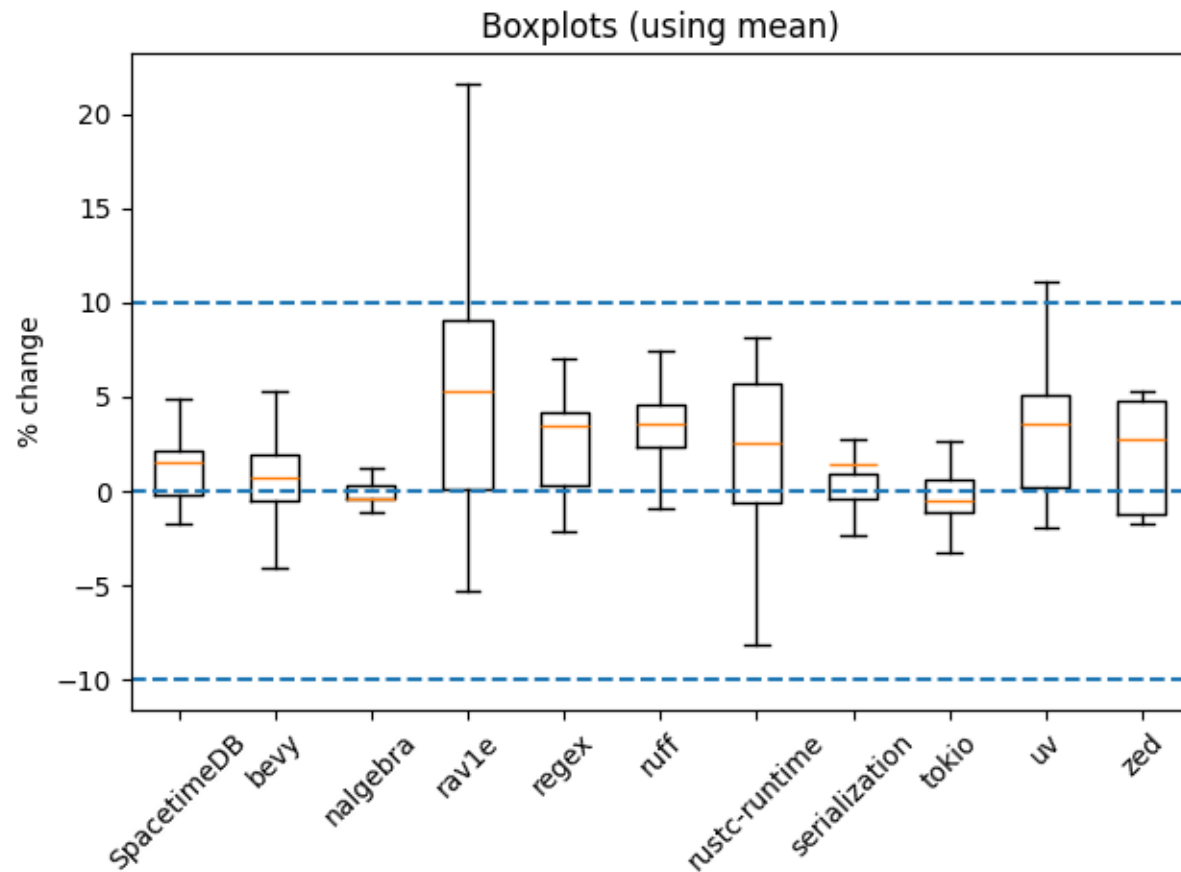
- Disabled bounds and arithmetic checks, overflow checks in stdlib containers, UTF-8 string checks
- Not counting overheads for redundant initializations (expecting $\geq 1\%$)

Slowdown in Hardened C++



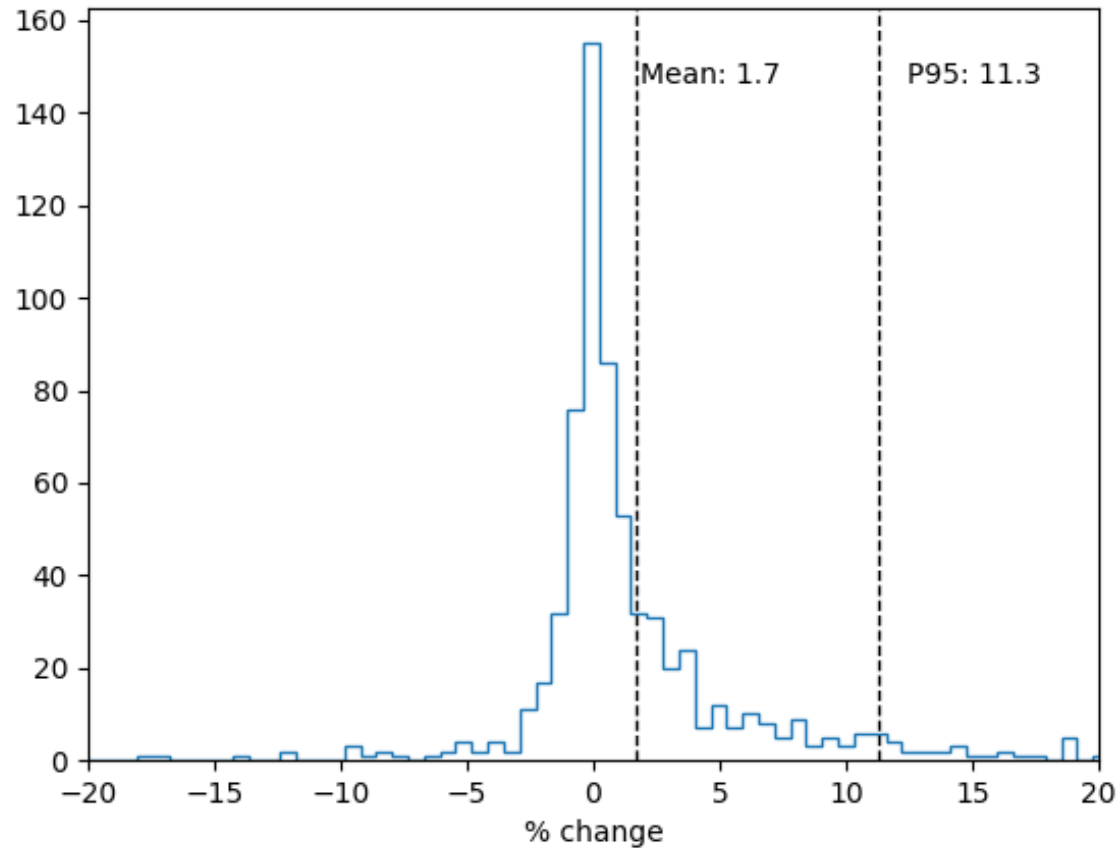
- Enabled hardening:
 - `-D_FORTIFY_SOURCE=3`
 - `-fsanitize=bounds,object-size`
 - Hardened STL
 - (initialization not enabled to match Rust)
- Unlike Rust majority of benchmarks **have not been tuned** for Hardened C++ case

Distribution of overheads in benchmarks



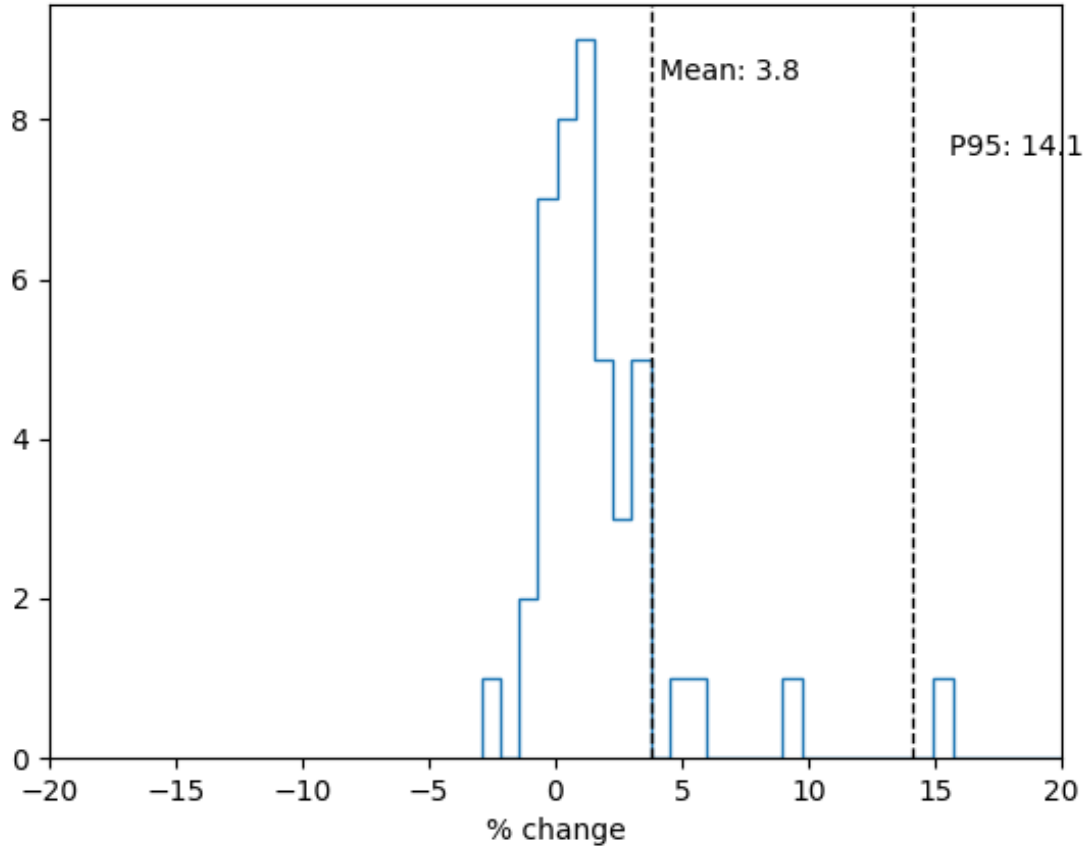
- Distribution of all benches in each project
- Similar tests with different parameters were averaged
- Data is **biased**:
 - Scenarios which are covered by many benches dominate
- Expecting at least +1% mean due to forced initialization

Distribution of overheads in benchmarks



- This graph is a pure illustration and does not prove anything!
- Distribution of all benches in all projects
- Similar tests with different parameters were averaged
- Data is **biased**:
 - Projects with larger numbers of benches dominate
 - Scenarios which are covered by many benches dominate
- Expecting at least +1% mean due to forced initialization

Distribution of overheads in Hardened C++ benchmarks



- This graph is a pure illustration and does not prove anything!
- Much fewer benchmarks than for Rust
- Unlike Rust majority of benchmarks **have not been tuned** for Hardened C++ case

Conclusions



- Rust community keeps balance between safety and performance:
 - Main focus is on memory safety guarantees
 - Other checks are enabled if they do not hurt performance
 - For example integer overflow errors are disabled by default
- Overhead of runtimes checks depends on the project
 - Usually under 5% on average (P95 11%) and comparable to Hardened C++
- Language features which affect performance the most:
 - Bounds checks, mandatory initialization, lack of inplace initialization
- These checks will most likely be further optimized in future versions of Rust and LLVM
- But their negative effects will never be completely avoided (at least in current language version)
- So they will always require workarounds (e.g. unsafe code) to reach optimal performance

What we didn't talk about ...



- [Codegen units](#)
 - Similar to unity builds in C/C++
- [Implementation of panics \(exceptions\)](#)
 - Similar to C++ exceptions
- [GC-sections by default](#)
- [ABI features](#)
- [Disabled PLT](#)
- [Implementation of virtual methods](#)
- [Stack Clashing checks](#)

The End

Many thanks to our kind reviewers:

- Vadim Petrochenkov
 - <https://github.com/petrochenkov>
- Alexander Chichigin
 - <https://github.com/Gabriel-Fallen>
- Jakub Beránek
 - <https://github.com/kobzol>
- Alexander Monakov
 - <https://github.com/amonakov>
 - <https://codeberg.org/amonakov>
- Per Vognsen
 - <https://mastodon.social/@pervognsen>



Latest version of this
presentation:

<https://github.com/yugr/rust-slides/blob/main/EN.pdf>

Appendix

Preparation of this talk

- We collected and analyzed over 350 papers, blogposts and discussions about the topic (see [materials](#))
- Overall time spent on preparation (Apr 2026):
 - > 3 man-months
- Analysis was conducted for
 - Rust 1.87.0 (May 2025)
 - Clang llvmorg-20.1.7 (June 2025)

Rust benchmarks

- Used standard Criterion benchmarks in each project (cargo bench), averaged with geomean
 - Some especially slow tests in oxipng were disabled
 - To counteract system noise we took minimum of average from several runs
 - Note that 1% geomean may mean that
 - All tests in project slowed down by 1%
 - 10% of tests slowed down by 10% (more likely)
 - Some tests could slow down (or speedup) by more than 10%
- Geomeans are not ideal
 - Biased by scenarios with more benches
- Further details [here](#)

C++ benchmarks

- Used Clang llvmmorg-20.1.7
- Llvm-bench ([benchmarks/cpp/llvm-bench](#))
 - Measured median time of -O2 compilation of CGBuiltin.cpp (the largest and slowest-to-compile file in LLVM) with Clang
 - Tested Clang was built with itself
- Ffmpeg-bench ([benchmarks/cpp/ffmpeg-bench](#))
 - Based on [Stack Smashing Protection and Its Performance Impact](#)
 - Measured median time to convert FLV to MP4
- Phoronix Test Suite ([benchmarks/cpp/PTS](#))
 - Popular benchmark suite from [phoronix-test-suite.com](#)
 - Measured botan, bullet, coremark, c-ray, dav1d, gcrypt, gmpbench, luajit, nginx, opencv, openssl, polybench-c, povray, rocksdb, simdjson, vpxenc, x265 (see [#55](#))
 - Used median time of benchmark
 - Ignored tests with Stdev > 0.5% or changes < 1%
- To counteract system noise we took minimum of average from several runs

Hardware

- All benchmarks (except no-static) were run on Intel Xeon E-2236
 - With fixed 2 GHz frequency
 - With Ubuntu 24.04 OS
- Servers have been set up according to [ulnit methodology](#)

TODO

- (Yugr) Benchmark meilisearch
- (Zak) Fast-math: update analysis (P3715, -fno-math-errno and -fno-trapping-math)
- (Zak) Struct reorder: fill analysis